

Walchand College of Engineering, Sangli

(Government aided Autonomous Institute)

Department of Computer Science and Engineering

TY CSE Mini Project 1(7CS345)

Report on

PromptOpt: Token-Efficient Prompt Optimization for LLMs

Submitted by

Piush U. Gogi	**255200005**
Nirzara A. Manade	**255200006**
Siddhi S. Bolaikar	**255200008**

Under the guidance of

Prof. A. S. Pawar

Department of Computer Science and Engineering,
Walchand College of Engineering, Sangli

2026 - 2027

Walchand College of Engineering, Sangli
(Government aided Autonomous Institute)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Project Report entitled, **"PROMPTOPT: TOKEN-EFFICIENT PROMPT OPTIMIZATION FOR LLMS"** submitted by **Piush U. Gogi, Nirzara A. Manade and Siddhi S. Bolaikar**, to Walchand College of Engineering Sangli, India, is a record of bonafide project work of course **Mini Project-I (7CS345)** carried out by them under my/our supervision and guidance and is worthy of consideration for the award of the degree of Bachelor of Technology in Computer Science & Engineering of the Institute.

Prof. A. S. Pawar	**Mr. H.R. Chandoba**		**Dr. A.R. SURVE**
Guide	**Mini Project Coordinator**	**External Examiner**	**Head Of Department**
Department of Computer Science and Engineering, Walchand College of Engineering, Sangli	Department of Computer Science and Engineering, Walchand College of Engineering, Sangli		Department of Computer Science and Engineering, Walchand College of Engineering, Sangli

Acknowledgement

We express our sincere gratitude to our guide, Prof. A. S. Pawar, for constant guidance, timely reviews and for pushing us to measure every claim instead of assuming it. The suggestions on evaluation, on the validation of our dataset and on presenting the system through one worked example shaped the way this project was carried out and reported.

We thank Mr. H. R. Chandoba, Mini Project Coordinator, for organising the mini-project reviews, and Dr. A. R. Surve, Head of the Department of Computer Science and Engineering, for providing the facilities and the environment to work on this project. We also thank the faculty member who reviewed a sample of our dataset, and the students who answered our survey.

Finally, we thank Walchand College of Engineering, Sangli, and our families and friends for their support throughout the project.

Declaration

We hereby declare that the work presented in this project report titled **"PROMPTOPT: TOKEN-EFFICIENT PROMPT OPTIMIZATION FOR LLMS"** submitted by us in the partial fulfillment of the requirement of the award of the degree of **Bachelor of Technology (B. Tech)** in the **Department of Computer Science & Engineering, Walchand College of Engineering, Sangli** is an authentic record of our project work carried out under the guidance of Prof. A. S. Pawar.

Date:

Place: Sangli

(Signature)

Piush U. Gogi
(255200005)
Nirzara A. Manade
(255200006)
Siddhi S. Bolaikar
(255200008)

Table of Contents

Abstract **12**

1 Introduction and Related work **13**

1.1 Background 13
1.2 Motivation and customer connect 13
1.3 Related work 14
1.4 Research gap 15
1.5 Organisation of the report 16

2 Problem statement **17**

2.1 Description 17
2.2 Formal statement 17
2.3 Scope of the work done 17
2.4 Alignment with the Sustainable Development Goals 18

3 Objectives **19**

4 Methodology **20**

4.1 Development approach (SDLC) 20
4.2 Phase 1: Requirement analysis 21
4.3 Phase 2: Dataset preparation 23
4.4 Phase 3: System design 29
4.5 Phase 4: Implementation of Stage A (feature detection) 31
4.6 Phase 4: Implementation of Stage B (rule-based optimization) 36
4.7 Phase 4: Implementation of Stage C (LoRA fallback) and its training 38
4.8 Phase 4: Rendering, token counting and the web application 43
4.9 Phase 5: Validation 44
4.10 Phase 6: Testing and evaluation method 44
4.11 Phase 7: Deployment 46
4.12 Technology stack 46

5 Project diagrams **47**

5.1 The example first 47
5.2 System architecture 47
5.3 Pipeline flow of one request 49
5.4 Database design 50
5.5 Evaluation flow 52

6 Test plan **53**

6.1 Unit testing 53
6.2 Integration testing 56
6.3 System testing 60

7 Results **61**

7.1 Results with screenshots and description 61
7.2 Comparative analysis: input, output and input + output tokens 69
7.3 Performance measures in tabular form and graphs 72
7.4 Result analysis 74

7.5 GUI images and screenshots....76
7.6 Screenshots of the GitHub project upload....78
7.7 Use of Git and GitHub....79
7.8 Challenges faced and how we solved them....80
8 Conclusion....81
Limitations of the proposed solution....81
Future scope and possible extensions....82
9 References....83
10 Scholarly contribution....85

List of Figures

Figure 4.1: SDLC phases followed in the project, with the two feedback loops....20
Figure 4.2: Project schedule (Gantt chart): all ten tasks done....21
Figure 4.3: Dataset pipeline: from Dolly-15k and CodeAlpaca-20k to PromptOpt Dataset v1.2 final.…23
Figure 4.4: The review example through every stage of PromptOpt....30
Figure 4.5: System architecture, with the review example under each component....30
Figure 4.6: Stage A: every arrow is labelled with what flows along it....32
Figure 4.7: Stage A scores for the review example: k-NN vote, head and final score against the 0.6 gate35
Figure 4.8: Stage B: rule order, the 0.6 confidence gate and the group fallback....36
Figure 4.9: Stage C in block form: routing, request, model, checker, fallback and the user's confirmation 39
Figure 4.10: Stage C training pipeline....40
Figure 5.1: The review example through the system....47
Figure 5.2: System architecture with the review example under each component....48
Figure 5.3: Module diagram: core modules, supporting modules and the data layer....48
Figure 5.4: Pipeline flow of one request, with the commit decision at step 7....50
Figure 5.5: Entity-relationship diagram (10 tables)....51
Figure 5.6: Evaluation flow: splits, variants and metrics....52
Figure 6.1: Correctness suite: correct answers per category, vague vs optimized....59
Figure 7.1: Improve prompt (simple app): the vague prompt, the pasted review and the target AI.....61
Figure 7.2: The improved prompt, written in ChatGPT's layout (50 tokens), with the task-type question62
Figure 7.3: Token usage analysis: both prompts on the same AI, 162 tokens instead of 270 (40% fewer)63
Figure 7.4: History (simple app): the prompt, the improved prompt and its token saving, kept for 30 days 64
Figure 7.5: Developer view, Optimize: the vague prompt and the pasted review, target GPT, category auto-detect....64
Figure 7.6: Stage A on the review example: summarization 0.52, below the 0.6 gate, so marked unsure 65
Figure 7.7: Stage B on the review example: two rules fired, each shown with the text before and after. .65
Figure 7.8: Stage C on the review example: the model filled format, constraints and category; the user is asked to confirm....66
Figure 7.9: The optimized prompt rendered for GPT, with 50 input tokens....67
Figure 7.10: Compare: the vague and the optimized prompt answered by the same model....68
Figure 7.11: Review example: input, output and total tokens on the same model....69
Figure 7.12: Input, output and input + output tokens per category, vague vs optimized....70
Figure 7.13: Mean input + output tokens per prompt by category, vague vs optimized....71
Figure 7.14: Benchmark split: quality, task success and input + output tokens....72
Figure 7.15: Stage A F1 per category (test split)....72
Figure 7.16: Test suite: correctness summary and the case grid....74
Figure 7.17: One case (cqa-01): gold answer 80300; the vague answer was wrong, the optimized right. 74
Figure 7.18: Results page: headline numbers, read from the final results file....75
Figure 7.19: A rejected Stage C answer: the checker failed, so Stage B's result is kept....75
Figure 7.20: Optimize with a coding example: B07, B05 and B03 fire; Stage C is skipped....76
Figure 7.21: Compare on the coding example: 248 tokens instead of 768 (−67.7%), both answers pass 6 of 6 tests....77
Figure 7.22: How it works: the pipeline explained inside the application....78
Figure 7.23: History: search, view and delete; prompts expire after 30 days....78
Figure 7.24: Image mode: the user's words are kept, missing attributes are suggestions....78
Figure 7.25: Status: providers with 24-hour usage, Stage C device, sandbox....78
Figure 7.26: GitHub repository piush365/PromptOpt: branches, tags and top-level folders....79

Figure 7.27: Release v1.0 with the Stage C adapter and the Stage A index.....79
Figure 7.28: Branches: main (default), ui-v2, correctness-suite.....79
Figure 7.29: Commits per week (all branches).....80

List of Tables

Table 1.1: Survey: how often users must ask an AI tool more than once to get the answer they want.....13
Table 1.2: Survey: what users notice about token consumption.....13
Table 1.3: Gap analysis: existing prompt tools vs PromptOpt.....15
Table 2.1: What the project covers.....17
Table 2.2: Sustainable Development Goals addressed by PromptOpt.....18
Table 4.1: SDLC phases: activities and outputs.....20
Table 4.2: Functional requirements.....22
Table 4.3: Non-functional requirements.....22
Table 4.4: Constraints and their consequences.....22
Table 4.5: Source datasets.....23
Table 4.6: Answer-length cut-offs per category.....24
Table 4.7: Automatic quality checks on every generated pair.....25
Table 4.8: Expansion plan per category.....26
Table 4.9: Purpose of each split.....27
Table 4.10: Rating design.....28
Table 4.11: Agreement of the three team raters on the 90 overlap rows.....28
Table 4.12: PromptOpt Dataset v1.2 final: rows per category and split.....29
Table 4.13: Fields of the intermediate representation.....31
Table 4.14: Main design decisions and their basis.....31
Table 4.15: Output of Stage A for the review example.....32
Table 4.16: How a neighbour's weight falls with its distance ($\tau = 0.05$).....33
Table 4.17: Keyword cues per category (weights in brackets).....33
Table 4.18: Confidence calculation for the review example.....34
Table 4.19: Rule-based detectors of Stage A.....35
Table 4.20: Stage B rule catalogue.....36
Table 4.21: Default output format added by B03 per category.....37
Table 4.22: Items Stage B may leave unresolved.....38
Table 4.23: Stage B steps for the review example.....38
Table 4.24: Fields Stage C is asked to fill.....39
Table 4.25: Base model Qwen2.5-0.5B-Instruct [10].....39
Table 4.26: Trainable LoRA parameters per transformer layer ($r = 16$).....40
Table 4.27: Stage C examples per split.....41
Table 4.28: Training settings.....42
Table 4.29: Validation loss during training.....42
Table 4.30: Layouts per target LLM.....43
Table 4.31: Decisions taken on the validation split.....44
Table 4.32: Technology stack.....46
Table 5.1: Main modules.....48
Table 5.2: Tables of the database.....51
Table 6.1: Types of testing and tools.....53
Table 6.2: Unit test cases per module and their pass rate.....54
Table 6.3: Unit and module test cases: four sample cases per module.....54
Table 6.4: Correctness suite: closed_qa (correct answers: vague 8/10, optimized 10/10).....57
Table 6.5: Correctness suite: information_extraction (correct answers: vague 7/10, optimized 10/10).....57
Table 6.6: Correctness suite: classification (correct answers: vague 7/10, optimized 8/10).....58
Table 6.7: Correctness suite: summarization (correct answers: vague 7/10, optimized 5/10).....58
Table 6.8: Correctness suite: coding (correct answers: vague 8/10, optimized 9/10).....58
Table 6.9: Correctness suite: accuracy and tokens per category (OpenAI's GPT model gpt-oss-120b).....59

Table 6.10: Browser end-to-end tests (Playwright, real application).....60
Table 7.1: Headline results on held-out data (output tokens measured on OpenAI's GPT model gpt-oss-120b).....69
Table 7.2: Mean tokens per prompt: input, output and input + output (482 test prompts, OpenAI's GPT model gpt-oss-120b).....70
Table 7.3: Change per prompt by token type (482 test prompts, OpenAI's GPT model gpt-oss-120b)….70
Table 7.4: Stage C ablation on the test split.....71
Table 7.5: Stage A per-category results (482 test prompts).....72
Table 7.6: Stage B per category (482 test prompts).....72
Table 7.7: Per-rule agreement with the dataset's optimized prompts (test).....73
Table 7.8: Stage C: zero-shot base model vs LoRA (964 test examples).....73
Table 7.9: Other components.....73
Table 7.10: Git and GitHub usage.....79
Table 7.11: Challenges and solutions.....80

Acronyms

API	Application Programming Interface
CI	Confidence Interval
CLIP	Contrastive Language-Image Pre-training
CPU	Central Processing Unit
ER	Entity-Relationship
GPU	Graphics Processing Unit
IR	Intermediate Representation
JSON	JavaScript Object Notation
k-NN	k-Nearest Neighbours
LLM	Large Language Model
LoRA	Low-Rank Adaptation
ORM	Object-Relational Mapping
PEFT	Parameter-Efficient Fine-Tuning
PII	Personally Identifiable Information
QA	Question Answering
SD	Stable Diffusion
SDLC	Software Development Life Cycle
SRS	Software Requirements Specification
UI	User Interface
bwrap	bubblewrap (Linux sandboxing tool)

Abstract

People write prompts for large language models (LLMs) the way they type a search query: short, vague and without saying what the answer should look like. The model then guesses, often producing long, unrequested answers that cost output tokens, time and money and are harder to use. PromptOpt is a lightweight, self-hosted system that rewrites such a prompt into a clear, structured and token-efficient one for the model the user works with (GPT, Gemini or Claude), explains every change, and measures whether the rewrite helps.

PromptOpt works in three stages. Stage A detects what a prompt is missing, Stage B fixes it with fifteen small, individually tested rules, and Stage C, a 0.5-billion-parameter model fine-tuned with LoRA, fills in only what the rules cannot resolve (6.4% of test prompts). The result is written in the layout each target LLM prefers and is served through a web application: a simple app for everyday users (improve a prompt, analyse its token usage on a real model, a 30-day history) and a developer view that shows every stage.

On 482 held-out test prompts, measured on OpenAI's GPT model gpt-oss-120b, optimized prompts reduce total tokens (input + output) by 39.2% (95% CI 35.9-42.5%): input grows slightly (226 to 241 tokens) while output falls from 682 to 165 tokens. An output format is stated in 95.0% of prompts instead of 3.1%; on the benchmark split, quality rises from 8.2 to 9.0 and task success from 67% to 85%. On 50 new hand-written cases with verifiable answers, optimized prompts were correct in 42 cases versus 37 for the vague prompts. The system is covered by 720 unit test cases (60 for each of 12 modules) and 16 browser tests, all passing.

Keywords: *prompt optimization, large language models, token efficiency, rule-based NLP, LoRA, FastAPI.*

1. Introduction and Related work

1.1 Background

Large language models such as GPT, Gemini and Claude are now used daily by students, developers and small teams for answering questions, summarising notices, sorting messages and writing code. Their cost and speed depend on tokens. A token is a piece of a word; every token of the prompt (the input) and every token of the answer (the output, including hidden reasoning tokens of reasoning models) is counted, billed on paid plans and charged against the daily budget on free plans. When the budget is used up, the free tier stops working for the rest of the day.

A large part of this cost is caused by the prompt itself. A prompt such as “hey can you just tell me if this review is good or bad”, typed together with a short product review, does not say what the answer should look like. The model therefore writes a heading, a list of positive and negative points and a closing paragraph. In our measurement, OpenAI's GPT model gpt-oss-120b answered this vague prompt with 166 output tokens. The optimized prompt produced by PromptOpt adds one line, “Provide only the label.”, and the same model answered with 41 output tokens; the total for the request fell from 270 to 162 tokens (−40%) and the answer came back in 97 ms instead of 353 ms. This review example is followed through every stage of the system in this report (Chapter 4 and Chapter 5).

1.2 Motivation and customer connect

Before designing the system we surveyed students, developers and members of small teams, the people who need more value from every token. The survey asked how often they had to ask an AI tool more than once to get the answer they wanted and what they noticed about token limits.

Table 1.1: Survey: how often users must ask an AI tool more than once to get the answer they want

Very often	Often	Sometimes	Rarely	Never
20%	30%	25%	15%	10%

Table 1.2: Survey: what users notice about token consumption

Observation	Share of respondents
Run out of tokens very quickly	72%
Need multiple prompts to get one output	58%
Notice a big drop in quality as tokens run low	46%

Observation	Share of respondents
Have to stop their work due to token limits	31%

Half of the respondents need to repeat a request often or very often, and almost three quarters run out of tokens quickly. A tool that makes the first prompt clear enough to get a usable, short answer addresses both problems at once: fewer repeated requests and fewer output tokens per request.

1.3 Related work

We studied four lines of work that are close to this problem: automatic prompt optimization, cost-aware prompt optimization, prompt compression and the sensitivity of LLMs to prompt format. We also used two technical building blocks: parameter-efficient fine-tuning and sentence embeddings.

Automatic prompt optimization

Ramnath et al. [1] survey automatic prompt optimization and define it as a search over the space of prompts for a fixed task, scored on a labelled development set. Representative methods are APE (Zhou et al. [4]), which asks an LLM to propose candidate instructions and keeps the best scoring one, and OPRO (Yang et al. [5]), which uses an LLM as an optimizer that sees previous candidates and their scores. These methods improve a task prompt that will be reused many times, but each optimization costs many LLM calls, needs labelled examples of the task and returns a prompt without explaining why it is better. They do not fit a single user who types a single, one-off prompt and wants the improved version immediately.

Cost-aware prompt optimization

CAPO (Zehle et al. [3]) adds the length of the prompt to the objective of an evolutionary search, so that shorter prompts with the same accuracy are preferred. It is the closest work to our goal of token efficiency, but it still optimizes one task prompt with many LLM evaluations, and it counts input length only. It is too expensive for a lightweight, self-hosted tool that must answer in well under a second.

Prompt compression

LLMLingua (Jiang et al. [6]) removes tokens from long prompts with a small language model to reduce input cost. Our measurements show that for short, everyday prompts the opposite lever matters: the optimized prompt is slightly longer (input 226 to 241 tokens on average), and the

saving comes from much shorter answers (output 682 to 165 tokens). An evaluation that counts only input tokens would miss this effect entirely, which is why we measure input and output tokens separately everywhere.

Sensitivity to prompt format

Ngweta et al. [2] show that LLM output quality changes with the formatting of the prompt alone, even when the content is the same. This motivates our renderers: the same content is laid out in the style each model family's own prompting guide recommends (XML tags for Claude, Markdown sections for GPT, plain labels with the instruction first for Gemini).

Building blocks

- Sentence-BERT (Reimers and Gurevych [11]) produces sentence embeddings whose cosine similarity reflects meaning; Stage A uses the small all-MiniLM-L6-v2 model of this family to find training prompts similar to a new prompt.
- LoRA (Hu et al. [7]) freezes a pretrained model and trains small low-rank matrices next to its weight matrices. With LoRA, fine-tuning the 0.5-billion-parameter Qwen2.5 model [10] fits on a 4 GB laptop GPU.
- The source datasets are Databricks Dolly-15k [8] (question answering, information extraction, classification and summarization) and CodeAlpaca-20k [9] (coding).

1.4 Research gap

Table 1.3 compares PromptOpt with four existing prompt tools that we examined in the requirement phase. All of them rewrite prompts, but none evaluates the rewritten prompt on a real model, none reports the token effect of input and output together, and none combines deterministic rules with a fine-tuned fallback.

Table 1.3: Gap analysis: existing prompt tools vs PromptOpt

Feature	PO.tools	Aiven	SiteGPT	BenchLM	PromptOpt
Rewrite prompts	Yes	Yes	Yes	Yes	Yes
Goal / framework customization	Limited	Yes	Yes	Limited	Yes
Preserve constraints	No	-	-	Yes	Yes
LLM output evaluation	No	No	No	No	Yes
Token efficiency report	-	Yes	-	-	Yes
Multi-stage pipeline	No	No	No	No	Yes
Comprehensive reporting	No	No	No	Partial	Yes

Feature	PO.tools	Aiven	SiteGPT	BenchLM	PromptOpt
Access	Free	Free	Free	Limited	Self-hosted

From the literature and the tool comparison we identified four gaps, each of which PromptOpt addresses:

1. No automated evaluation of optimized prompts against the original prompts on a real model. PromptOpt's Compare feature and its evaluation harness run both prompts on the same model and report tokens, latency, quality and correctness.
2. Limited or unclear handling of constraints and requirements. PromptOpt keeps every word of the user's data, adds only missing constraints and logs every change with the text before and after it.
3. No systematic measurement of token efficiency with its quality trade-off. PromptOpt measures input and output tokens separately on a held-out test split and reports quality and correctness next to them.
4. No multi-stage pipeline that combines rule-based and learned methods. PromptOpt uses deterministic rules first and a small LoRA model only where the rules stop.

1.5 Organisation of the report

Chapter 2 states the problem and Chapter 3 the objectives. Chapter 4 describes the methodology phase by phase of the software development life cycle (SDLC) we followed: requirement analysis, dataset preparation, design, implementation of the three stages including the training of Stage C, validation, testing and deployment. Chapter 5 collects the project diagrams, starting from the review example. Chapter 6 is the test plan (unit, integration and system testing), Chapter 7 the results with screenshots, and Chapter 8 concludes with limitations and future scope.

2. Problem statement

Problem definition: to build a lightweight, self-hosted system that rewrites a vague user prompt into a clear, structured prompt for the user's chosen LLM, shows every change it makes, and measures whether the rewritten prompt reduces total tokens (input + output) without lowering the quality of the answer.

2.1 Description

Users often write prompts that are vague, verbose or missing key details: the output format, a length, the programming language, the labels to choose from, or which text is meant by “this”. The model then guesses, which wastes output tokens, increases latency and gives answers that are inconsistent and harder to use. The problem addressed in this project is to detect these gaps in a single user prompt, fix them without changing what the user asked for, write the result in the layout the target model prefers, and prove with measurements on held-out data that the fix helps.

2.2 Formal statement

Given a user prompt ‘p’, a target LLM ‘t’ ∈ {GPT, Gemini, Claude}, a category ‘c’ (one of five, or auto-detect) and an optional attachment type ‘a’, produce an optimized prompt ‘p′’ rendered for ‘t’ such that:

1. the user's intent and every piece of data in ‘p’ are preserved;
2. the missing output format and relevant constraints are stated;
3. every change from ‘p’ to ‘p′’ is logged with the text before and after it;
4. the total tokens (input + output) needed to answer ‘p′’ are lower than for ‘p’, without lowering answer quality or correctness, as measured on a held-out test split.

2.3 Scope of the work done

Table 2.1: What the project covers

Aspect	Covered in this project
Task categories	closed_qa, information_extraction, classification, summarization, coding, plus “other”
Target LLMs (rendering)	GPT, Gemini and Claude layouts; exact token counts for GPT, approximate for the others

Aspect	Covered in this project
Attachments	image, PDF, PPTX, DOCX, spreadsheet, code file, other: as a modifier, not a category
Optimizer	Stage A detection, Stage B rules, Stage C LoRA fallback, offline (no API needed)
Evaluation	input, output and total tokens, quality, task success, correctness, ablation; answers from OpenAI's GPT model gpt-oss-120b on free tiers
Application	FastAPI + React web app: a simple app (Improve prompt, Token usage analysis, History of 30 days) and a developer view (Optimize, Compare, Test suite, Results, Image mode)

2.4 Alignment with the Sustainable Development Goals

Table 2.2: Sustainable Development Goals addressed by PromptOpt

Goal	How PromptOpt contributes
SDG 9: Industry, innovation and infrastructure	an open, self-hosted tool that makes everyday use of LLMs clearer and cheaper, built only on free and open components
SDG 12: Responsible consumption and production	fewer tokens per answer (39.2% fewer on the test split) means less computing, and so less energy, for every request

3. Objectives

The objectives below describe what was built and measured in this project.

1. To detect what a user prompt is missing (task category with a confidence, output format, length, tone, audience, programming language, filler words and ambiguous references) using spaCy, regular expressions and a sentence-embedding classifier (Stage A).
2. To fix the detected gaps with fifteen deterministic, individually unit-tested rules that log every change with the text before and after it (Stage B).
3. To fine-tune a small language model (Qwen2.5-0.5B-Instruct with LoRA) on a 4 GB laptop GPU and use it only for what the rules cannot resolve, under a validated contract with a fallback to Stage B (Stage C).
4. To build PromptOpt Dataset v1.2 final, 5,184 vague-to-optimized prompt pairs in five categories, validated by automatic checks and human raters, with train, validation, test and benchmark splits that share no instruction.
5. To represent the optimized prompt in one model-agnostic intermediate representation and render it for GPT, Gemini and Claude, with token counts per target.
6. To support an optional attachment type (image, PDF, PPTX, DOCX, spreadsheet, code file) as a modifier.
7. To measure input, output and total tokens, quality, task success and correctness on held-out data with a real model, together with an ablation study of the three stages.
8. To provide a working web application: a simple app (improve a prompt, compare the token usage of the original and the optimized prompt on the same model, a history with 30-day retention) and a developer view that shows every stage, with removal of e-mail addresses and phone numbers before storage and one database transaction per request.

4. Methodology

This chapter describes how PromptOpt was built, phase by phase. Each phase of the software development life cycle (SDLC) is described with its activities, the decisions taken and their basis, and the output it handed to the next phase. Throughout the chapter one running example is followed through the system: the prompt “hey can you just tell me if this review is good or bad” typed together with the review “The battery lasts two days and the camera is great, but the phone heats up while gaming.”

4.1 Development approach (SDLC)

The project followed an iterative waterfall model: the phases run in order, each with a clear output, but two feedback loops are allowed. Results on the validation split feed back into implementation (rule wording, thresholds, the Stage C policy), and problems found during implementation feed back into dataset preparation (the v1.1 repair and the v1.2 expansion). A strict rule separated tuning from testing: every decision was taken on the validation split, the code was then frozen, and the test split was run only once.

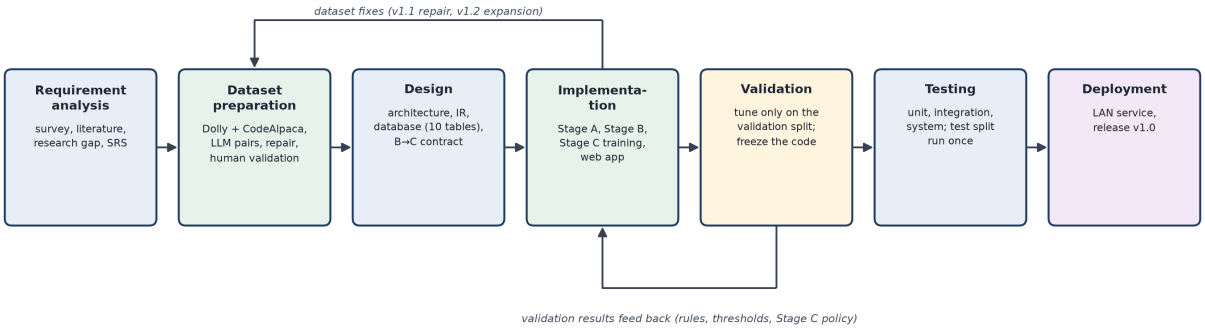


Figure 4.1: *SDLC phases followed in the project, with the two feedback loops*

Table 4.1: SDLC phases: activities and outputs

Phase	Main activities	Output
1. Requirement analysis	customer survey, literature review, research gap, functional and non-functional requirements (SRS)	requirements, scope, five fixed categories
2. Dataset preparation	source selection, cleaning, sampling, LLM generation of prompt pairs, automatic checks, repair, splits, human validation	PromptOpt Dataset v1.2 final (5,184 pairs)
3. Design	architecture, intermediate representation (IR), B-to-C contract, database schema (10 tables), API	design documents, diagrams

Phase	Main activities	Output
4. Implementation	Stage A, Stage B rules, Stage C data and LoRA training, renderers, FastAPI back end, React UI	working system
5. Validation	tuning on the validation split only; freeze of the code; freeze check	frozen pipeline
6. Testing	unit, integration and system tests; evaluation on the test split (once) and the benchmark	test reports, final numbers
7. Deployment	service on the laptop reachable on the local network; release v1.0 with the trained artefacts	running application

The work was planned as ten tasks over four months, from problem identification and customer connect in July to the final demo and documentation in October. Requirement and design tasks overlap in July and August; prototype development, model development (rules and LoRA) and evaluation follow from September. All ten tasks are done.

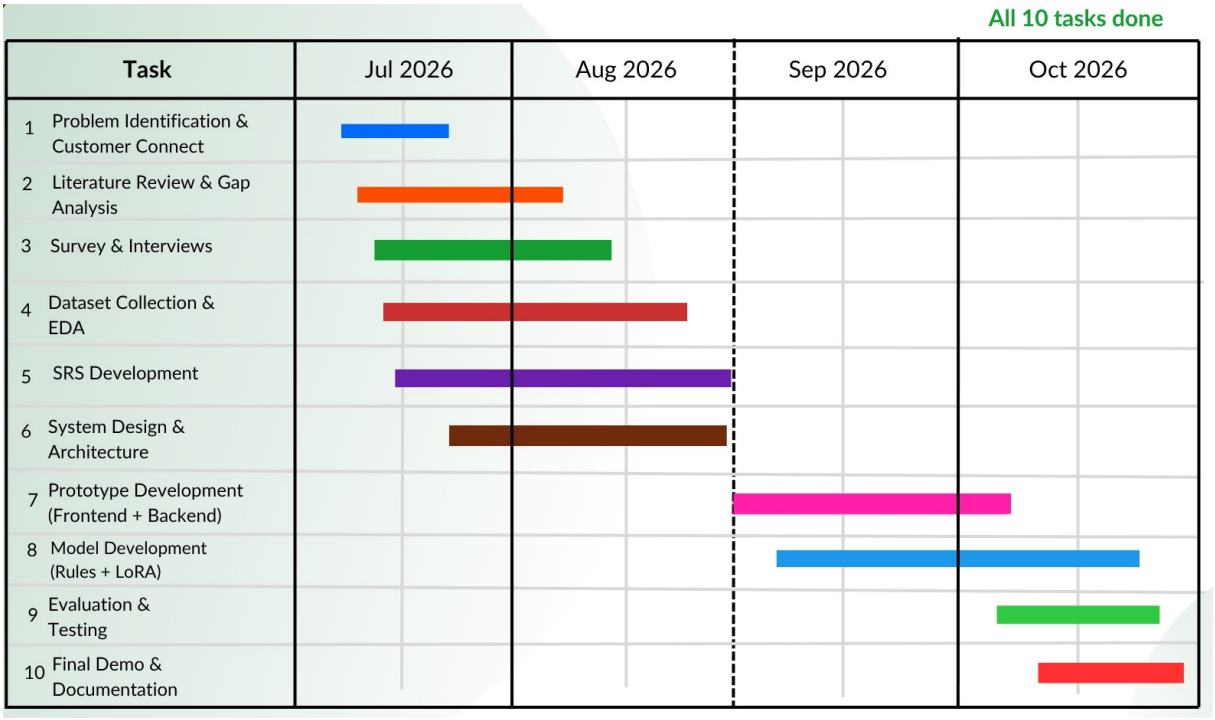


Figure 4.2: *Project schedule (Gantt chart): all ten tasks done*

4.2 Phase 1: Requirement analysis

The requirement phase combined three inputs: the customer survey (Section 1.2), the literature review and research gap (Sections 1.3 and 1.4), and the constraints of the environment the system had to run in. The requirements were written down in a Software Requirements Specification (SRS) before any code was written.

Functional requirements

Table 4.2: Functional requirements

ID	Requirement
FR1	The user enters any prompt and optionally pastes the text the prompt is about.
FR2	The user selects a target LLM (GPT, Gemini or Claude), a category (one of five, or auto-detect) and optionally an attachment type.
FR3	The system detects missing parts of the prompt and shows them to the user.
FR4	The system adds only what is missing, keeps the user's data verbatim and shows every change with the text before and after it.
FR5	A user-selected category overrides the automatic one; when the system is unsure, it asks the user.
FR6	The optimized prompt is shown in the layout of the chosen target LLM, with the number of input tokens.
FR7	Token usage analysis (Compare in the developer view) sends the original and the optimized prompt to the same real model and shows both answers, tokens and time.
FR8	A history lists, searches, shows and deletes past prompts; prompts expire after 30 days.

Non-functional requirements

Table 4.3: Non-functional requirements

ID	Requirement	How it was met
NFR1	Self-hosted and offline optimizer	Stages A, B and C run locally; no API is needed to optimize
NFR2	Explainable	every rule is a small pure function; every change is logged and shown
NFR3	Fast	Stage A on CPU in milliseconds; Stage C under 3 s per prompt on the laptop GPU
NFR4	Privacy	e-mail addresses and phone numbers are removed before storage; prompts expire after 30 days
NFR5	Reliable storage	one database commit per request; a failure stores nothing
NFR6	Honest measurement	tuning on the validation split only; the test split run once; the answering model always named
NFR7	Testable	every rule and detector has unit tests; the whole suite runs without network

Constraints that shaped the design

Table 4.4: Constraints and their consequences

Constraint	Consequence in the design
One laptop GPU with 4 GB of memory (NVIDIA RTX 3050) and a CPU-only default install	Stage C uses a 0.5-billion-parameter model with LoRA; the application also runs fully without it
Only free API tiers (Groq: 200,000 tokens per day per model; Cerebras: 1,000,000)	every LLM call is recorded in a rolling 24-hour ledger; tools stop at 70% of a limit and resume the next day
Examiners must be able to follow every change	rules are small functions; every change is logged with before/after text

Constraint	Consequence in the design
Results must not be tuned on the test data	split discipline, a frozen code version and a byte-level freeze check
Privacy of users' prompts	personal data removed before anything is stored; 30-day retention

4.3 Phase 2: Dataset preparation

No public dataset pairs a vague prompt with a clear version of the same request, so we built one: PromptOpt Dataset. Each row is a pair of a degraded prompt (how a rushed, non-expert user types a request) and an optimized prompt (the same task stated clearly, with an explicit output format and only helpful constraints), together with the original instruction, the context passage and the reference answer of the source row. The dataset is used to train Stage A's classifier, to build Stage C's training data, and to evaluate every stage. It is stored outside the database and outside git.

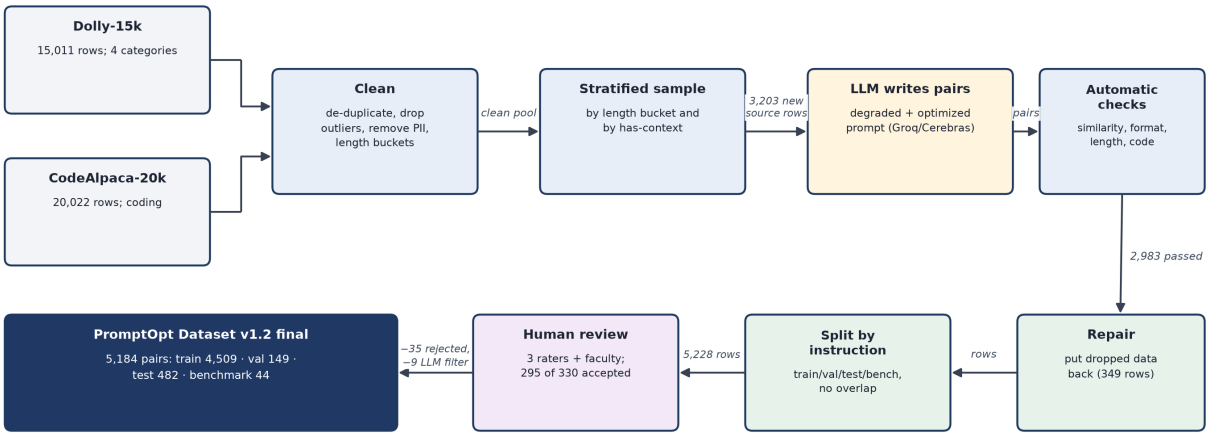


Figure 4.3: *Dataset pipeline: from Dolly-15k and CodeAlpaca-20k to PromptOpt Dataset v1.2 final*

4.3.1 Sources

Table 4.5: Source datasets

Source	Rows	Used for
Databricks Dolly-15k [8]	15,011	closed_qa (1,773), information_extraction (1,506), classification (2,136), summarization (1,188); brainstorming and creative writing as “other” examples for Stage A only
CodeAlpaca-20k [9]	20,022	coding

Every source row keeps a permanent identifier (its row number in the original file, e.g. dolly-4205), assigned before any filtering, so a row can always be traced back to its source even after the dataset is rebuilt.

4.3.2 Cleaning

1. Category filter: Dolly rows of the four categories with a non-empty instruction and response.
2. De-duplication on the normalised instruction and context (lower case, whitespace collapsed); the first copy is kept.
3. Removal of reference answers that are extremely long for their category (rule below).
4. Removal of e-mail addresses and phone numbers with the same patterns the application uses.
5. CodeAlpaca: de-duplication, and outputs of fewer than three words dropped (they are not code).
6. Attributes added to every row: whether it has a context, whether it states a format, word counts, and a length bucket (short, medium or long: the tertile of instruction + context length within the category).

A Dolly row ‘i’ in category ‘c’ is dropped when its reference answer is longer than the category's cut-off:

$$cutoff(c) = median(w) + 3 \cdot \sigma(w), \quad drop\ row\ i\ if\ w(i) > cutoff(c)$$

where ‘w’ is the number of words in the reference answer and σ the sample standard deviation. The median is a robust centre for right-skewed word counts, and three standard deviations is the classical outlier tail; the rule removes a few very long answers that would otherwise dominate the evaluation.

Table 4.6: Answer-length cut-offs per category

Category	Rows after de-dup	Median words	σ	Cut-off	Dropped
closed_qa	1,759	17	48.3	162.0	36
information_extraction	1,501	19	105.1	334.4	45
classification	2,133	14	36.4	123.3	37
summarization	1,186	54	154.5	517.5	10

4.3.3 Generation of prompt pairs

For each sampled source row, an LLM (gpt-oss-120b or gpt-oss-20b on Groq and Cerebras, qwen3.8-27b as a fallback) wrote both prompts in one JSON answer. The instructions to the generator describe the degraded prompt (short, casual, no format, may refer to "this") and the optimized prompt, with requirements per category: closed_qa answers only from the text and states a length; extraction states what to extract and the output structure; classification lists the labels and ends with the items; summarization states a length and a focus; coding names the language, behaviour, inputs and outputs. A data rule requires that any data written inside the instruction (items to classify, code, numbers, the named subject) appears verbatim in both prompts. Settings: temperature 0.7 for variety, JSON output, context truncated to 1,000 characters, up to three attempts per row.

To use the free daily quotas efficiently, five rows were sent per call (one shared system prompt). On the same ten rows this reduced the cost from about 1,070 to 400 tokens per row with checks as good or better; every reply echoes the row's identifier and results are matched only by identifier, never by position.

4.3.4 Automatic quality checks

Similarity between two texts is the cosine of their all-MiniLM-L6-v2 embeddings; since the embeddings are normalised to length 1, it is their dot product:

$$sim(a, b) = e(a) \cdot e(b) = \cos\theta, \text{ between } -1 \text{ and } 1$$

A generated pair is rejected if any of the following flags is set:

Table 4.7: Automatic quality checks on every generated pair

Flag	Condition	Why
degraded drifted	sim(original, degraded) < 0.45	the degraded prompt must ask the same thing
not degraded	degraded equals the original (or sim > 0.98) and the original stated a format	nothing was degraded
degraded keeps a format	the format patterns match the degraded prompt	degraded prompts have no format
degraded longer	words(degraded) − words(data) > 1.25 × words(original) + 3	degraded prompts are short
optimized drifted	sim(original, optimized) < 0.35, or no shared key word and sim < 0.50	the optimized prompt keeps the task
coding target not code	coding row whose optimized prompt names no code word	–

Flag	Condition	Why
optimized too long	words(optimized) − words(data) > 120	over-specification

4.3.5 Repair of dropped data (v1 to v1.1)

The first generation often dropped the literal data of the instruction. For example, the original "Identify which instrument is string or percussion: Sheker, Taishogoto" became an optimized prompt that asked to classify "each listed instrument" without listing them, so it could not be answered. A deterministic repair finds the data in the original instruction (an item list, code on later lines, an arithmetic expression), decides whether a prompt lost it, and appends it verbatim. For non-classification prompts, data is considered lost when less than half of its content words are still present:

$$coverage = |content\ words\ of\ data\ also\ in\ prompt| \,/\, |content\ words(data)|, \quad lost\ if\ coverage < 0.5$$

Optimized prompts that stopped naming the instruction's subject ("the temple" instead of "Doleshwor Mahadeva temple") were sent back to the generator for a minimal edit, accepted only if the edit names every lost subject and adds at most 12 words. Result: 349 optimized and 20 degraded prompts repaired, 75 subjects restored, 9 rows removed because their data existed nowhere; every change is written to a log file.

4.3.6 Expansion to about 1,000 rows per category (v1.2)

To reach about 1,000 rows per category, new rows were generated only from source rows never used before. The number to sample in category 'c' accounts for the share of rows that survive the checks:

$$needed(c) = max(0,\ 1000 - rows(c)), \quad to_sample(c) = min(unused(c),\ ceil(needed(c) \,/\, rate(c) \times 1.05))$$

where rate(c) is the survival rate of the first generation and 1.05 adds a 5% margin. Rows were drawn by stratified sampling over length bucket and has-context, with the largest-remainder method so that every stratum keeps its share.

Table 4.8: Expansion plan per category

Category	v1.1 rows	Survival rate	Needed	Unused source rows	Sampled
closed_qa	425	0.850	575	1,223	711
information_extraction	449	0.898	551	956	645

Category	v1.1 rows	Survival rate	Needed	Unused source rows	Sampled
classification	474	0.948	526	1,596	583
summarization	436	0.872	564	676	676 (all)
coding	472	0.944	528	18,524	588
all					3,203

Of the 3,203 generated pairs, 2,983 passed the checks and 2,972 were added after repair, giving v1.2 with 5,228 rows.

4.3.7 Splits

Rows are split by original instruction, so the same instruction (or a paraphrase of it) never appears in two splits. Each category has 100 test rows, 30 validation rows and 10 benchmark rows; the rest is training data. The extra test rows of v1.2 came only from new rows that no tuning had seen.

Table 4.9: Purpose of each split

Split	Used for	Who may look at it
train	Stage A's index and classifier head; Stage C's training data	every phase
validation	every tuning decision (thresholds, rule wording, Stage C policy, judge wording)	development
test	the final offline numbers of Stage A, B and C and the full token evaluation	run once, after the code was frozen
benchmark	the final run with a real model and a blind judge	run once

4.3.8 Human validation

The three team members and the faculty guide rated rows with five yes/no questions. A row is accepted only if all five answers are yes.

1. Q1: the degraded prompt asks for the same task as the original instruction.
2. Q2: the degraded prompt is a realistic, vaguer version a real user might type.
3. Q3: the optimized prompt keeps the task and intent, without adding facts, answers or unwanted requirements, and without dropping data.
4. Q4: the optimized prompt is clearer and better specified than the degraded one.
5. Q5: the category label is correct.

Table 4.10: Rating design

Who	Rows	Purpose
all three team members	90 overlap rows (18 per category)	agreement between raters
each team member alone	60 extra rows (test and benchmark splits and borderline rows first)	filtering bad pairs
each team member alone	20 rows of the new v1.2 generation	human check of the new rows
faculty guide	20 of the overlap rows	independent check (never decides a row)

Overlap rows are decided by majority vote per question (2 of 3); other rows by their single rater. In total 330 distinct rows were rated: 295 accepted and 35 rejected. The faculty guide accepted 19 of 20 rows.

Agreement on the overlap rows is reported with Fleiss' kappa [14], Gwet's AC1 [12] and the raw agreement. With ‘n’ = 3 raters and n(i,j) the number of raters giving answer j to item i, the agreement on item i and the mean over the I items are

$$P(i) = [\Sigma\, n(i,j)^2 - n] / [n(n-1)], \quad \bar{P} = (1/I)\, \Sigma\, P(i)$$

Kappa compares $\bar{P}$ with the agreement expected by chance from the answer frequencies p(j), while AC1 uses a chance term that stays small when one answer dominates:

$$\kappa = (\bar{P} - P_e) / (1 - P_e), \; P_e = \Sigma\, p(j)^2; \quad AC1 = (\bar{P} - P_e') / (1 - P_e'), \; P_e' = 2\pi(1-\pi)$$

Worked example for Q1: 81 items were answered yes by all three raters, 8 split 2-1 and 1 split 1-2, with 260 yes answers out of 270. Then $\bar{P} = (81 + 9/3)/90 = 0.933$, p(yes) = 0.963, $P_e = 0.963^2 + 0.037^2 = 0.929$, so $\kappa = (0.933 - 0.929)/(1 - 0.929) = 0.065$, while $AC1 = (0.933 - 0.071)/(1 - 0.071) = 0.928$. Because almost every answer is yes, the chance term of kappa approaches 1 and a handful of disagreements decides its value; this is the known kappa paradox [13]. We therefore report kappa and AC1 together.

Table 4.11: Agreement of the three team raters on the 90 overlap rows

Question	% yes	All three agree	Fleiss' κ	Gwet's AC1
Q1 degraded asks the same task	96.3%	90.0%	0.065	0.928
Q2 degraded is realistic	96.3%	88.9%	−0.038	0.920
Q3 optimized keeps the intent	96.3%	90.0%	0.065	0.928
Q4 optimized is better	98.1%	94.4%	−0.019	0.962
Q5 category is correct	98.1%	94.4%	−0.019	0.962
accept (all five yes)	86.3%	67.8%	0.092	0.719

After the human review, an LLM-assisted filter re-read the 90 overlap rows as a strict reviewer, without seeing the human answers. Only rows where the optimized prompt adds facts, leaks the answer, sets an impossible constraint or changes the task were removed: 9 rows. The filter never overrides a human answer and is not part of any agreement statistic.

4.3.9 Final dataset

PromptOpt Dataset v1.2 final = 5,228 rows − 35 rows rejected by the raters − 9 rows removed by the filter = 5,184 rows.

Table 4.12: PromptOpt Dataset v1.2 final: rows per category and split

Category	Train	Validation	Test	Benchmark	Total
closed_qa	951	29	99	10	1,089
information_extraction	867	30	93	10	1,000
classification	906	30	98	8	1,042
summarization	920	30	94	6	1,050
coding	865	30	98	10	1,003
all	4,509	149	482	44	5,184

Known limitations of the dataset: the optimized prompts are written by an LLM and only 330 rows were rated by people; Dolly's category labels are noisy in the question / extraction / summary group; and some CodeAlpaca reference answers are wrong (these are listed and excluded from the evaluation summaries).

4.4 Phase 3: System design

The design starts from the user's request and follows it through three stages that each do one job. Figure 4.4 shows the running example as it passes through the whole system; the rest of this chapter explains each block.

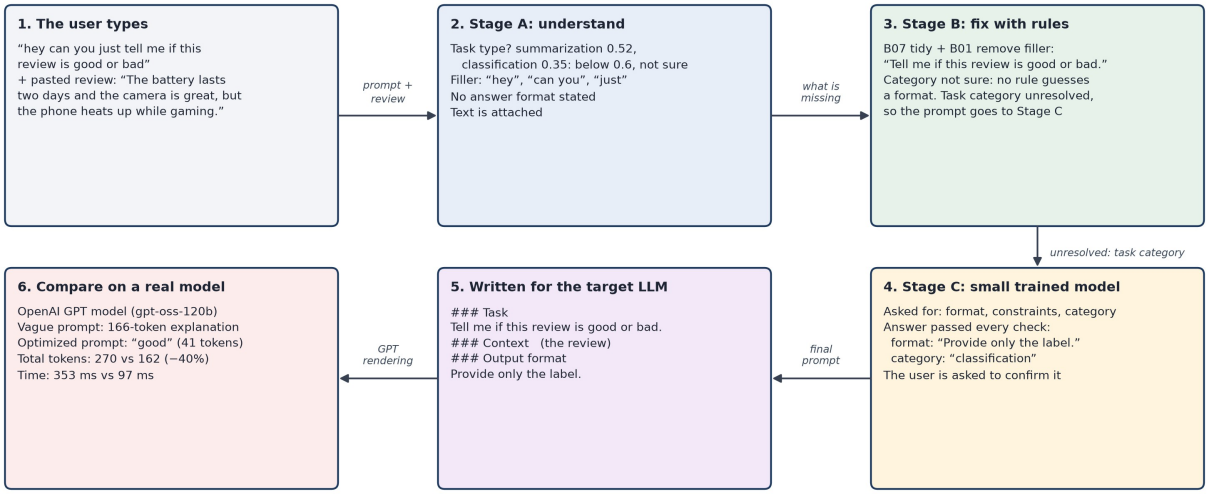


Figure 4.4: *The review example through every stage of PromptOpt*

Stage A reads the prompt and writes down what is missing; it never changes the prompt. Stage B turns the prompt into a structured intermediate representation (IR) and applies deterministic rules to it. Stage C, a small fine-tuned model, is called only when Stage B leaves the task category or an ambiguous reference unresolved. The final IR is written out by a renderer for the chosen LLM, and Compare can run the original and the optimized prompt on the same real model.

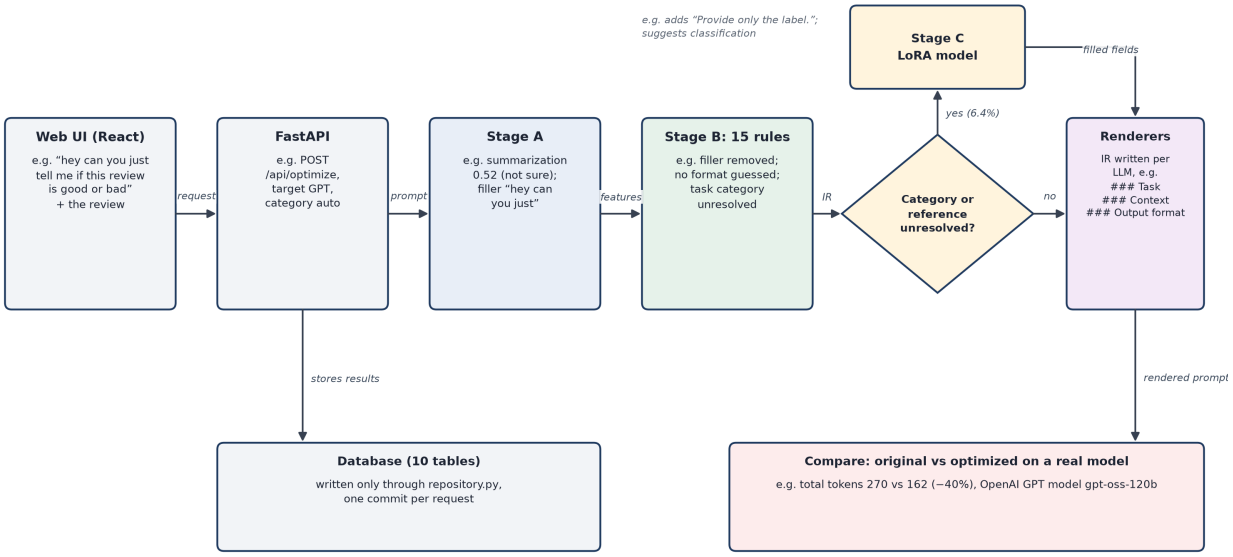


Figure 4.5: *System architecture, with the review example under each component*

4.4.1 The intermediate representation (IR)

All stages work on one model-agnostic data structure instead of editing a string. Rules return a new IR instead of changing the old one, so every change is small, testable and can be logged.

Table 4.13: Fields of the intermediate representation

Field	Meaning	Review example after Stage C
task	what to do, in the user's words, cleaned	Tell me if this review is good or bad.
context / context_ref	data moved out of the prompt; where the material is	the pasted review (separate)
constraints	length, programming language, grounding	(none)
requirements	task-specific rules: allowed labels, how to use an attachment	(none)
output_format	how to lay out the answer	Provide only the label.
attachment	attachment type and optional file name	none
category / category_source	category used and where it came from	unresolved: user asked to confirm “classification”
target_llm	the user's target	GPT
unresolved	what Stage B could not fix	task category (waiting for the user)

4.4.2 Design decisions

Table 4.14: Main design decisions and their basis

Decision	Basis
Rules first, model last	rules are cheap, explainable and testable; a model is used only where rules cannot decide (6.4% of test prompts)
Category rules only above a confidence of 0.6	a wrong format is worse than none; on the validation split Stage A was right 77-96% of the time above 0.6 and only 47-59% between 0.4 and 0.6
Stage C fills only requested fields	everything else is locked, so the model cannot damage what the rules already fixed
Stage C's category is only a suggestion	on validation prompts routed for the category, Stage A was right 40% and Stage C 30% of the time: neither is reliable, so the user decides
One IR, several renderers	the content is decided once; only the layout differs per model, and tests prove no field is lost
One repository module and one commit per request	a failure half-way stores nothing; privacy rules are applied in one place

4.5 Phase 4: Implementation of Stage A (feature detection)

Stage A turns the raw prompt into a PromptFeatures object. It combines a learned classifier for the task category with rule-based detectors for everything else.

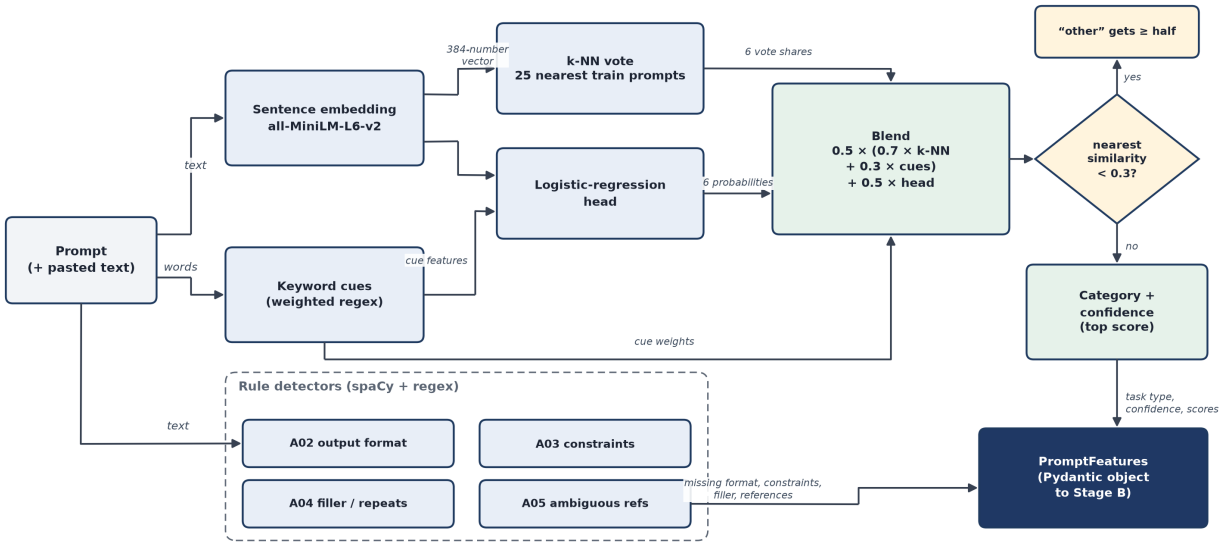


Figure 4.6: *Stage A: every arrow is labelled with what flows along it*

Table 4.15: Output of Stage A for the review example

Field	Meaning	Review example
task_type, confidence	the category with the highest score, and that score	summarization, 0.52
category_scores	the score of every category (they sum to 1)	summarization 0.52, classification 0.35, other 0.09, extraction 0.03, coding 0.01, closed_qa 0.00
has_format_spec (A02)	an output format is stated	no
missing_constraints (A03)	constraints relevant to the category that are not stated	length, audience, tone
redundant_phrases (A04)	filler and repeated text	“hey”, “can you”, “just”
ambiguous_refs (A05)	references to material that is not there	none (the review is pasted)
has_context	material to work on was pasted or embedded	yes

4.5.1 Sentence embedding and the labelled index

The prompt is encoded by all-MiniLM-L6-v2, a six-layer transformer that maps a sentence to a vector of 384 numbers, normalised to length 1. The model is small (about 90 MB) and fast on a CPU: 7.9 ms per prompt on average. The labelled index is built once from the training split only: for each of the 4,509 training rows, its degraded prompt and its original instruction, labelled with the row's category (9,018 texts), plus 500 out-of-scope instructions (Dolly brainstorming and creative writing) labelled “other”. Validation and test prompts never enter the index.

4.5.2 k-nearest-neighbour vote

For the query vector q, the similarity to every index vector e(i) is s(i) = e(i) · q. The k = 25 most similar examples vote for their category, each with a weight that falls quickly with its distance from the nearest one (a softmax with temperature τ = 0.05):

$$w(i) = \exp((s(i) - s_{max}) / \tau), \quad knn(c) = \Sigma\, w(i) \text{ over neighbours of category } c \,/\, \Sigma\, w(i) \text{ over all 25}$$

Table 4.16: How a neighbour's weight falls with its distance (τ = 0.05)

$s_{max} - s(i)$	0.01	0.02	0.05	0.10	0.20
weight w(i)	0.819	0.670	0.368	0.135	0.018

4.5.3 Keyword cues

Hand-written regular expressions with weights add explicit evidence. A cue counts once if it matches anywhere in the prompt; the raw score of a category is the sum of its matching weights, normalised over the categories.

Table 4.17: Keyword cues per category (weights in brackets)

Category	Cues
coding	programming terms such as function, script, array, regex (1.0); a language name such as Python, Java (1.5); “write/create/build … function/program/script/query” (2.0)
summarization	summary, summarize, TL;DR, sum up, recap, gist, condense (3.0); “main/key points” (1.5)
classification	classify, categorize, label, sort into, sentiment (3.0); “which of these is” (1.5); “tell me which” (2.0); “X or Y:” (1.0)
information_extraction	extract (3.0); “list/pull out/find … names/dates/places” (2.0); “mentioned in” (1.5); “list all” (1.5)
closed_qa	starts with a question word (1.5); ends with “?” (1.0); “according to the text” (1.5)

4.5.4 Logistic-regression head

A multinomial logistic regression is trained on the same index: its input is the 384-number embedding plus six keyword features (five normalised cue scores and whether the prompt ends with a question mark), and it outputs a probability for each of the six categories:

$$p(c \mid x) = \exp(w(c) \cdot x + b(c)) \,/\, \Sigma \exp(w(c') \cdot x + b(c')), \quad x \text{ has 390 numbers}$$

It is trained with L2 regularisation (C = 4.0, chosen by 5-fold cross-validation on the training split) and balanced class weights, so that the 500 “other” examples count as much in total as about 1,800 examples of each task category.

4.5.5 Confidence calculation

The confidence of Stage A is the final score of the winning category. The final score of every category combines the three signals:

$$blend(c) = 0.7 \cdot knn(c) + 0.3 \cdot kw(c) \quad (\text{if any keyword cue fired; otherwise } blend(c) = knn(c))$$

$$score(c) = 0.5 \cdot blend(c) + 0.5 \cdot p(c \mid x), \qquad confidence = \max_c score(c)$$

If even the nearest example is not similar ($s_{max} < 0.3$), nothing in the data looks like the prompt: every score is multiplied by keep = $s_{max} / 0.6$ and "other" receives the rest, so "other" always wins. The weights (0.7 / 0.3 and 0.5 / 0.5), k = 25, $\tau = 0.05$ and the 0.3 threshold were tuned on the validation split. In 5-fold cross-validation on the training split the blend reached 72.9% accuracy, against 69.3% for the k-NN vote alone and 70.4% for the head alone.

Worked example. For the review prompt no keyword cue fires ("good or bad" is not followed by a colon or question mark), so blend(c) = knn(c). The table shows the calculation; Figure 4.7 shows it as a chart.

Table 4.18: Confidence calculation for the review example

Category	k-NN vote	Head	0.5 · k-NN + 0.5 · head
closed_qa	0.000	0.001	0.001
information_extraction	0.050	0.003	0.026
classification	0.690	0.008	0.349
summarization	0.181	0.858	0.520
coding	0.019	0.010	0.014
other	0.060	0.120	0.090

The similar training prompts mostly vote for classification (0.69), while the head, which learned that "tell me … this review" often asks for a summary, prefers summarization (0.86). The blend picks summarization with a confidence of 0.52. Because 0.52 is below the 0.6 gate, Stage B will not trust this category, which is the right decision here: the prompt is actually a classification request. The nearest example has a similarity of 0.53, above 0.3, so the "other" backstop does not apply.

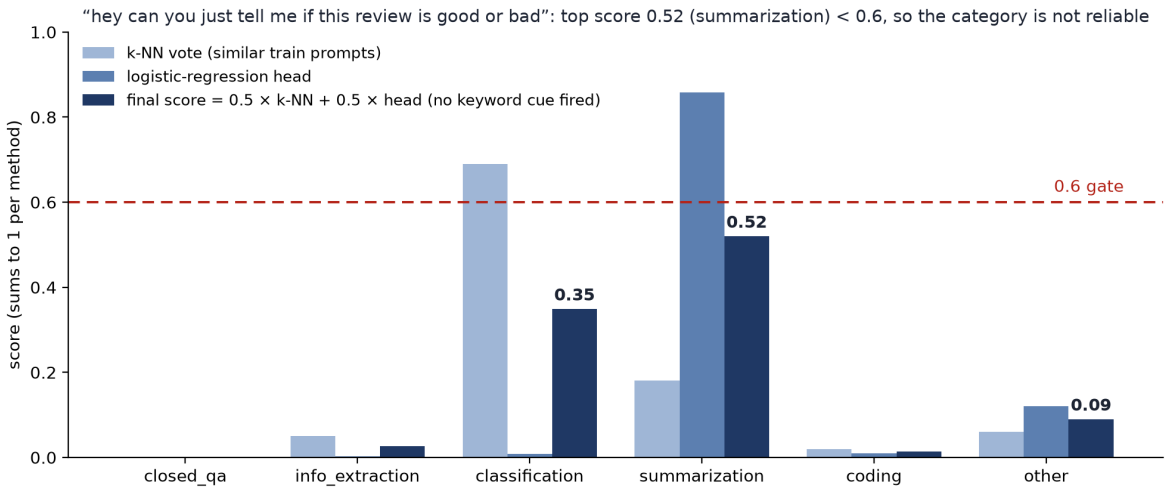


Figure 4.7: *Stage A scores for the review example: k-NN vote, head and final score against the 0.6 gate*

The confidence is a blended score, not a calibrated probability, which is why the 0.6 gate was chosen empirically on the validation split rather than derived from it.

4.5.6 Rule-based detectors (A02-A05)

Table 4.19: Rule-based detectors of Stage A

Detector	What it finds	Examples of patterns
A02 output format (26 patterns)	an explicitly stated output format	JSON, a table, a bulleted list, "in 3 sentences", "only the label", "yes or no"
A03 constraints	length, tone, audience, programming language	"brief", "formal tone", "explain to a beginner", "in Python"
A04 redundancy (17 patterns)	filler and repeated sentences or words	"hey", "can you", "just", "the the"
A05 ambiguous references	references to material that is not there (only when no text is given)	"summarize this", "the above", "fix it"

Which constraints are worth asking for depends on the category: a length for closed_qa; length, audience and tone for summarization; the programming language for coding; none for extraction and classification, whose output size is fixed by the data. For the review example (detected as summarization) A03 therefore reports length, audience and tone as missing; Stage B fills none of them because the category is below the gate. Each detector returns the exact words it matched, so the user interface can highlight them.

4.6 Phase 4: Implementation of Stage B (rule-based optimization)

Stage B fixes what fixed rules can fix. It creates the IR from the prompt, runs fifteen rules over it in a fixed order, logs every change with the text before and after it, and records what it could not fix.

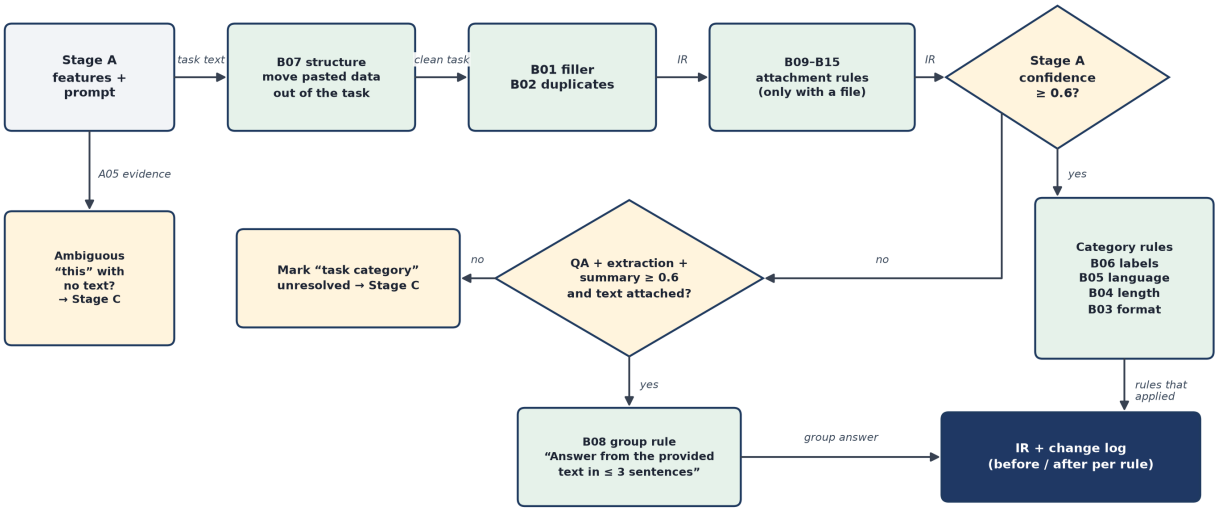


Figure 4.8: *Stage B: rule order, the 0.6 confidence gate and the group fallback*

4.6.1 Rule order

The rules always run in this order: B07 (structure) first, then B01 (filler) and B02 (duplicates), the attachment rules B09-B15, the group rule B08, and finally the category rules B06 (labels), B05 (language), B04 (length) and B03 (format). B07 runs first so that the user's data is moved out of the task before any other rule edits text; B03 runs last because it must know whether B08 already handled the prompt.

Table 4.20: Stage B rule catalogue

Code	Rule	Condition	Action	Example
B07	Structure	always; data moves only if clearly data	moves pasted data or code into its own block; capital letter, full stop	"classify these as fruit or vegetable: apple, carrot" → items in a separate block
B01	Remove filler	a filler phrase from A04	deletes it unless quoted or part of the content	"hey can you just tell me …" → "Tell me …"
B02	Remove duplicates	repeated sentence or doubled word	keeps one copy	"sort the the list" → "Sort the list"
B09-B15	Attachment	an attachment of that type is given	adds how to use it and "say so instead of guessing"	PDF: "Cite the page or section numbers …"

Code	Rule	Condition	Action	Example
B08	Group fallback	confidence < 0.6, text attached, QA + extraction + summary ≥ 0.6	adds what is missing of "Answer from the provided text in at most three sentences."	"what are the big inventions from berkeley in that text?"
B06	Labels	classification, reliable, no labels stated	states the allowed labels	"Use only these labels: "string", "percussion"."
B05	Language	coding, reliable, no language	"Use Python." or "Keep the language of the given code."	"write code to get all permutations of a string"
B04	Length	closed_qa or summarization, reliable, no length	"Answer in at most two sentences." / "Keep it under 100 words."	"summarize this article" + text
B03	Output format	no format stated; reliable; no group	adds the category's default format	coding: "Return only the code, in a single code block."

Table 4.21: Default output format added by B03 per category

Category	Format added
closed_qa	Start with the direct answer.
information_extraction	List each extracted item on its own line; if the text does not contain it, reply "Not found".
classification	For each item, output "item: label" on its own line (or "Output only the label." for a single item).
summarization	Use bullet points.
coding	Return only the code, in a single code block.

4.6.2 The 0.6 gate and the group fallback B08

Rules B03-B06 add category-specific content (a format, a length, a language, labels). A classification layout on a summarization request would mislead the model, so these rules act only when the category is reliable: Stage A's confidence is at least 0.6, or the user chose the category. Below the gate, Stage A often cannot tell closed_qa, information_extraction and summarization apart, but it is much surer that the prompt is one of the three. B08 therefore acts at group level when

$$\textit{max score} < 0.6 \quad \textit{and} \quad \textit{text attached} \quad \textit{and} \quad \textit{score(closed_qa)} + \textit{score(extraction)} + \textit{score(summarization)} \geq 0.6$$

and adds only what is missing of "Answer from the provided text in at most three sentences." When it was introduced, it reduced the share of validation prompts sent to Stage C from 52.0% to 11.3% without any new wrong-category additions. On the test split it fires on 149 of 482 prompts (30.9%).

4.6.3 Routing to Stage C

Table 4.22: Items Stage B may leave unresolved

Unresolved item	Set by	Sent to Stage C?
task category	category “other” or confidence < 0.6, not cleared by B08	yes
ambiguous reference	A05 found a reference and no attachment explains it	yes
output format	B03, when the category is not reliable	no (shown to the user)
label set	B06, when no labels could be found	no (shown to the user)

Stage B also stores a confidence for the user interface: the Stage A confidence minus 0.2 for every unresolved item (0.3 as the base for “other”), clipped to [0, 1]. It is not used for routing.

4.6.4 The review example in Stage B

Table 4.23: Stage B steps for the review example

Step	Text before	Text after
B07 structure	hey can you just tell me if this review is good or bad	Hey can you just tell me if this review is good or bad.
B01 filler	Hey can you just tell me if this review is good or bad.	Tell me if this review is good or bad.
B08	not applied: the group score (0.00 + 0.03 + 0.52 = 0.55) is below 0.6	–
B03-B06	not applied: confidence 0.52 < 0.6	–

Stage B leaves two items unresolved, the task category and the output format. The task category routes the prompt to Stage C.

4.7 Phase 4: Implementation of Stage C (LoRA fallback) and its training

Stage C is Qwen2.5-0.5B-Instruct with a LoRA adapter that we trained. It fills only the IR fields that Stage B left unresolved; every other field is locked. Its answer is checked by the same detectors as the rest of the pipeline, and if the check fails, the prompt keeps Stage B's result.

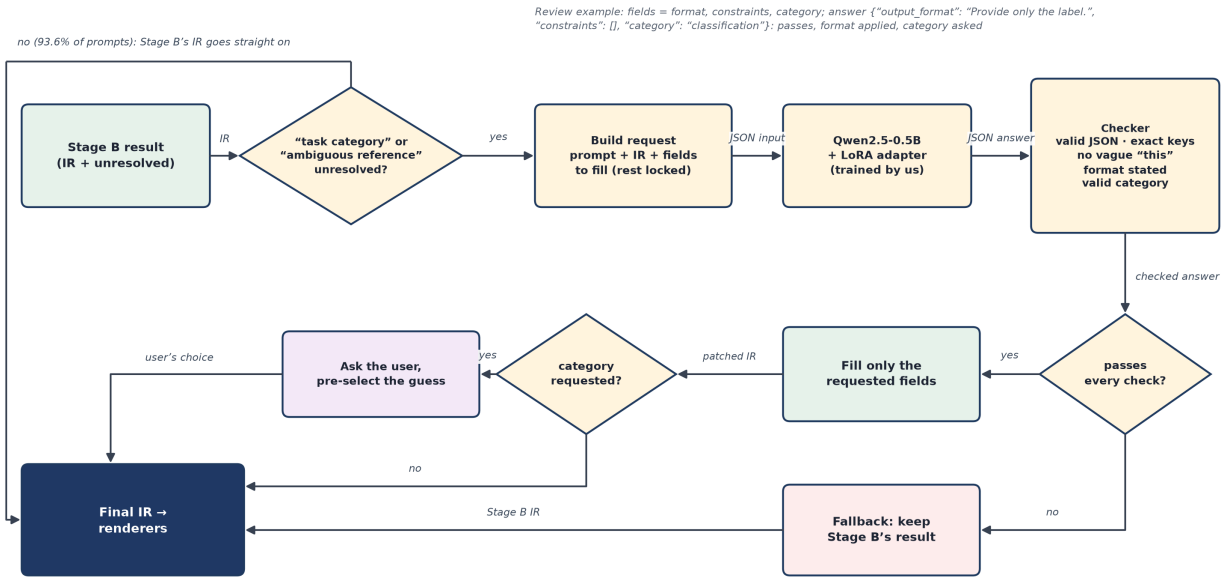


Figure 4.9: *Stage C in block form: routing, request, model, checker, fallback and the user's confirmation*

4.7.1 When Stage C runs and what it may fill

Table 4.24: Fields Stage C is asked to fill

Unresolved item	Fields requested	Reason
task category	output_format, constraints, category	Stage B added nothing category-specific, so format and constraints are missing too
ambiguous reference	task	the task must be rewritten to be self-contained

The model receives a short system prompt and the request as JSON: the user's prompt, Stage A's category and confidence, whether text is attached, the attachment type, Stage B's IR with the unresolved fields set to null, and the list of fields to fill. The pasted text itself is not sent: Stage C writes the instruction, not the answer, and this keeps the input short. For the review example the request asks for output_format, constraints and category, and the model answered:

```
{"output_format": "Provide only the label.", "constraints": [], "category":
"classification"}
```

4.7.2 Base model and LoRA

Table 4.25: Base model Qwen2.5-0.5B-Instruct [10]

Property	Value
parameters	about 494 million
transformer layers	24
hidden size	896
attention	14 query heads of 64 dimensions; 2 key/value heads (grouped-query attention)

Property	Value
feed-forward	SwiGLU, 896 → 4,864 → 896
vocabulary	151,936 tokens
weights	bfloat16

LoRA [7] freezes every base weight W_0 and learns a low-rank update next to it:

$$h = W_0 x + (\alpha / r) \cdot B\,A\,x, \quad A: r \times d_in \text{ matrix}, \; B: d_out \times r \text{ matrix}$$

A is initialised randomly and B with zeros, so at the start the model equals the base model. We used rank r = 16, α = 32 (scale 2) and dropout 0.05 on all attention and feed-forward projections. A LoRA pair on a d_in → d_out projection has r(d_in + d_out) parameters:

Table 4.26: Trainable LoRA parameters per transformer layer (r = 16)

Projection	Shape	r (d_in + d_out)
q_proj	896 → 896	28,672
k_proj	896 → 128	16,384
v_proj	896 → 128	16,384
o_proj	896 → 896	28,672
gate_proj	896 → 4,864	92,160
up_proj	896 → 4,864	92,160
down_proj	4,864 → 896	92,160
per layer		366,592

Over 24 layers this gives 8,798,208 trainable parameters, 1.78% of the model, stored in a 34 MB file. At inference the update is merged into the weights ($W = W_0 + (\alpha/r)BA$), so Stage C costs the same as the base model.

4.7.3 Training data

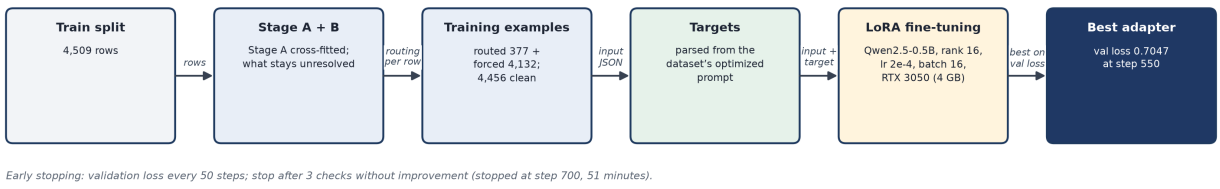


Figure 4.10: *Stage C training pipeline*

One training example is built per dataset row. Stage A is run on the training rows with cross-fitting: the rows are split into five folds by instruction, and each fold is classified by an index built without it. Otherwise every training prompt would find itself as its own nearest neighbour and Stage A would look unrealistically accurate (cross-fitted accuracy on train: 73.9%, close to 74.7% on test). Stage B is then run for real, and the target is parsed out of the dataset's optimized prompt.

Table 4.27: Stage C examples per split

Kind	Rule	Train	Validation	Test
routed	Stage B really sends the row to Stage C: its natural fields	377	10	31
forced	every other row: a random non-empty subset of {task, output_format, constraints}	4,132	139	451
forced_all	validation and test only: every row once more with all three fields	–	149	482
examples		4,509	298	964

Only 8.4% of training rows are routed naturally, too few to learn from, so forced examples teach the model to fill any requested field while respecting the locked ones. Each field is kept independently with probability 0.5 (an empty draw is replaced by one field), so an example asks for one field with probability 1/2, two with 3/8 and three with 1/8. 53 examples whose target was not cleanly separable were left out, giving 4,456 training examples.

The targets come from a parser that splits the dataset's optimized prompt into IR fields, reusing Stage A's detectors so that every sentence goes to exactly one field: the first sentence is the task; later sentences go to output_format if they state a structure, to constraints if they state a length, tone, audience, language or grounding, and otherwise stay in the task. The task was separated in 100% of rows, the output format in 94.0% and constraints in 75.0%; a hand check of 20 random parses found 17 fully correct.

4.7.4 Training procedure

The model sees the system prompt and the request, and learns to produce the JSON answer. The loss is the mean cross-entropy over the answer tokens only (the prompt tokens are masked), because the model must learn to write the JSON, not to repeat its input:

$$L(\theta) = -\frac{1}{|T|} \sum_{t \in T} \log p_\theta(x(t) \mid x(<t)), \quad t \text{ over the answer tokens } T$$

Table 4.28: Training settings

Setting	Value	Reason
effective batch	16 examples per step (micro-batch 1 × 16 accumulation)	fits the 4 GB GPU with the same effective batch
optimizer	AdamW [18], learning rate 2×10^{-4}, weight decay 0	usual LoRA learning rate
schedule	linear warm-up for 25 steps, then linear decay to 0	avoids large early updates
epochs	at most 3, with early stopping	stop when validation loss stops improving
gradient clipping	L2 norm ≤ 1.0	stability
precision and memory	bf16 autocast, LoRA weights in fp32, gradient checkpointing	fits 4 GB
sequence length	≤ 512 tokens (longest example 446)	no example dropped
seed	13	reproducibility

Every 50 steps the validation loss (token-weighted mean over all validation examples) is computed; the adapter is saved when it improves, and training stops after three evaluations without improvement.

Table 4.29: Validation loss during training

Step	50	100	200	300	400	500	550	600	700
val loss	0.852	0.789	0.745	0.726	0.716	0.706	0.705	0.744	0.761

The best validation loss, 0.7047, was reached at step 550 (a perplexity of e^0.7047 = 2.02 per answer token). After the third epoch began, the validation loss rose for three evaluations while training continued, the usual sign of over-fitting, and training stopped at step 700 after 51 minutes on the RTX 3050 laptop GPU.

4.7.5 Checker, fallback and category policy

Stage C answers with greedy decoding (at most 192 new tokens). The answer is accepted only if all checks pass:

1. it parses as one JSON object;
2. its keys are exactly the requested keys;
3. a rewritten task has at least two words and A05 finds no ambiguous reference in it;
4. an output format, if present, states a format according to A02;
5. constraints are a list of non-empty strings;

6. a category is one of the five categories.

Any failure keeps Stage B's result. For example, for “hey can you just summarize this article for me” without the article, Stage C answered “Summarize the provided text …”, which still refers to a text that is not there, so it was rejected. When the category was requested, Stage C's category is never applied on its own: the user interface asks the user and pre-selects Stage C's suggestion, while the format and constraints are applied. For the review example all checks passed, “Provide only the label.” was applied, and the user was asked to confirm “classification”.

Without fine-tuning, the base model passed these checks in only 5.0% of test examples (only 51.2% of its answers had exactly the requested keys); with the LoRA adapter, 98.2% passed.

4.8 Phase 4: Rendering, token counting and the web application

The final IR is written out for the chosen model family with the same four sections everywhere (task, context, constraints, output format), laid out the way each provider's prompting guide recommends.

Table 4.30: Layouts per target LLM

Target	Layout	Review example (output format section)
Claude	XML tags; the context first, then task, constraints, output format	<output_format> Provide only the label. </output_format>
GPT	Markdown sections ### Task / ### Context / ### Constraints / ### Output format	### Output format Provide only the label.
Gemini	plain labels, the instruction first and the context last	Output format: Provide only the label.

Data blocks are wrapped in a code fence one backtick longer than any backtick run inside the data, so pasted code cannot close the block early. Tests parse every rendering back and check that no field is lost. Input tokens are counted exactly for GPT with the tiktoken o200k_base tokenizer; for Claude and Gemini, whose tokenizers are not available offline, they are estimated as characters divided by 4 and labelled as approximate (on the test renderings this estimate is within about 5-8% of the GPT count on average). For the review example the GPT rendering has 50 input tokens against 33 for the user's prompt alone.

The web application is a FastAPI back end serving a React and TypeScript front end with two views. The simple app at / is for everyday users and uses no pipeline terms: Improve prompt (type the prompt, optionally paste the text it is about, pick ChatGPT, Gemini or Claude, copy the

improved prompt), Token usage analysis (both prompts sent to the same real model) and History (30 days); Section 7.1.1 shows it. The developer view at /dev is for monitoring: Optimize shows each stage in turn (Section 7.1.2); Compare, History, Test suite, Results, How it works, Image mode and Status complete it. All database access goes through one repository module (Chapter 5).

4.9 Phase 5: Validation

Every tuning decision was taken on the validation split (149 rows), never on the test split. The table below lists the main decisions and the validation evidence behind them.

Table 4.31: Decisions taken on the validation split

Decision	Validation evidence
category gate at 0.6	Stage A right 77-96% of the time at confidence ≥ 0.6, 47-59% between 0.4 and 0.6
group fallback B08	prompts routed to Stage C fell from 52.0% to 11.3%, no new wrong-category additions
Stage A settings (k, τ, weights, threshold)	tuned on validation accuracy (75.8% on validation)
Stage C only for routed prompts	forcing every prompt through Stage C lowered task intent from 0.855 to 0.773 on validation
Stage C category only as a suggestion	on routed validation prompts Stage A was right 40% and Stage C 30% of the time
single-label classification format	an earlier rule wrongly picked it for 8 validation prompts with several items; made stricter

After the last decision, the code was frozen with a git tag. From then on no Stage A or Stage B rule, threshold or judge setting was changed; bugs found later were reported, not fixed. Features added afterwards had to prove that they change nothing on the frozen pipeline: a freeze check runs Stage A and Stage B of the frozen and the current code on all 482 test prompts, writes every feature, IR, text, change and routing decision as JSON, and compares the SHA-256 hashes of the two dumps. A single changed character anywhere changes the hash; the two hashes are identical.

4.10 Phase 6: Testing and evaluation method

Testing has two parts: functional tests of the software (unit, integration and system tests, Chapter 6) and the evaluation of the optimizer's effect on real answers (Chapter 7). All answers in the evaluation come from OpenAI's GPT model gpt-oss-120b on the Groq or Cerebras free tier, at temperature 0, with at most 2,048 output tokens and reasoning effort “low”. Input and output token counts are the provider's own counts; output tokens include the model's hidden reasoning tokens,

as billed. Claude and Gemini were not run, so all output-token numbers in this report are for the GPT model only.

Token reduction

For a prompt ‘i’ with b(i) tokens before (vague prompt) and a(i) tokens after (optimized prompt), for input, output or total tokens:

$$r(i) = 100 \cdot (b(i) - a(i)) / b(i), \quad \text{mean reduction} = (1/n)\, \Sigma\, r(i)$$

$$\text{aggregate reduction } R = 100 \cdot (\Sigma\, b(i) - \Sigma\, a(i)) / \Sigma\, b(i)$$

The mean of the per-prompt reductions (39.2% for total tokens) counts every prompt equally and is the headline; the aggregate reduction (55.4%) weights prompts by their size and is larger, because prompts with long unrequested answers shrink most.

Confidence interval and significance

The 95% confidence interval of the mean reduction is a percentile bootstrap [19] with 10,000 resamples of the per-prompt reductions. Significance of the paired token counts is tested with the Wilcoxon signed-rank test [20], which uses only the signs and ranks of the paired differences and so suits heavy-tailed token counts. For small paired yes/no outcomes (correct or not) the exact McNemar test is used on the discordant pairs:

$$p = \min(1,\ 2 \cdot \Sigma \text{ from } i = 0 \text{ to } k \text{ of } C(n, i) \cdot 2^{\wedge}(-n)), \quad n = \text{wins} + \text{losses}, \quad k = \min(\text{wins}, \text{losses})$$

Quality, task success and correctness

- Quality: a blind judge from a different model family (qwen3.8-27b) scores each answer from 0 to 10 against the reference answer, without knowing which prompt produced it.
- Task success: checked automatically where possible (classification: every item gets the reference label; coding: a code block that parses; closed_qa: the judge's yes/no).
- Correctness suite: 50 new hand-written cases, 10 per category, each with one verifiable gold answer, checked deterministically or with hidden unit tests run in a sandbox.

Cost

The project ran on free tiers, so the money spent was zero. For a paid model with input price p(in) and output price p(out) per million tokens, cost = (T(in) · p(in) + T(out) · p(out)) / 10^6. Because the saving comes from output tokens, which usually cost several times more than input tokens, the

cost saving is larger than the token saving: with output priced 4 times input, the test-split means give a saving of 69.5%.

4.11 Phase 7: Deployment

The application runs on the development laptop as a user service (uvicorn app.api:app on port 8080), reachable on the local network with an optional password; Stage C runs on the laptop GPU and the application falls back to Stages A and B when the GPU or the adapter is not available. The trained Stage C adapter, the Stage A category index and the data manifest are published as a GitHub release (v1.0), so a fresh clone can be run by following the README. SQLite is used in development and PostgreSQL is supported for deployment by changing one setting.

4.12 Technology stack

Table 4.32: Technology stack

Layer	Technologies
Front end	React 19 + TypeScript, Vite, Tailwind CSS, Radix UI, Recharts; Playwright for end-to-end tests
Back end	Python 3.14, FastAPI, Uvicorn, Pydantic
Database	SQLAlchemy 2.0 ORM; SQLite (development), PostgreSQL (deployment)
NLP / features	spaCy, regular expressions, Sentence-Transformers (all-MiniLM-L6-v2), pyspellchecker
Model / fine-tuning	Hugging Face Transformers, PEFT (LoRA), PyTorch (CUDA), Qwen2.5-0.5B-Instruct
Tokenization	tiktoken (o200k_base)
LLM providers (live models)	Groq and Cerebras free tiers: OpenAI's gpt-oss-120b as the stand-in for GPT; qwen3.8-27b as judge; Gemini wired in (needs a key, not yet run)
Data	Dolly-15k and CodeAlpaca-20k, rewritten into prompt pairs with the Groq API
Testing / tooling	pytest, Playwright, bubblewrap sandbox, Git and GitHub

5. Project diagrams

This chapter collects the project diagrams. Following the guidance of the review, it starts with the running example and then shows the architecture, the modules, the flow of one request through the pipeline and the database.

5.1 The example first

A student pastes a short product review and types “hey can you just tell me if this review is good or bad”. The figure below follows this request: Stage A is not sure of the task type (summarization 0.52 against classification 0.35, below the 0.6 gate) and finds filler words and no answer format; Stage B removes the filler but does not guess a format; Stage C adds “Provide only the label.” and suggests classification, which the user confirms; the GPT rendering is sent to the model; and Compare shows that the request now costs 162 tokens instead of 270.

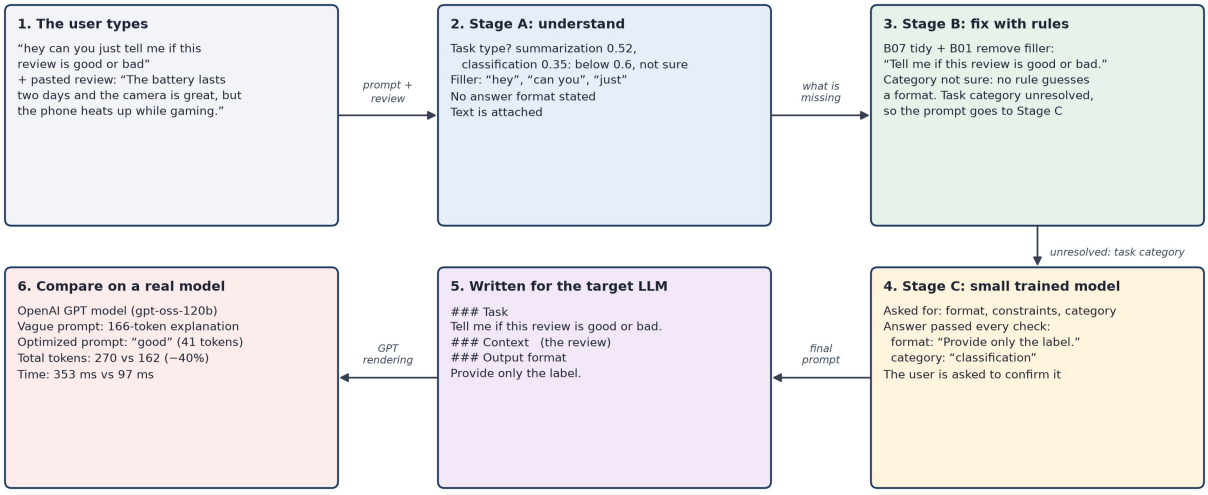


Figure 5.1: *The review example through the system*

5.2 System architecture

The architecture diagram shows the same example under each component. The React user interface sends the request to the FastAPI back end; Stage A, Stage B and (when needed) Stage C produce the IR; the renderers write it out for the target model; Compare (the token usage analysis in the simple app) sends both prompts to a live model, today Groq and Cerebras as stand-ins for GPT, with Gemini wired in but not yet run; and the database (SQLite or PostgreSQL) stores the results through one repository module.

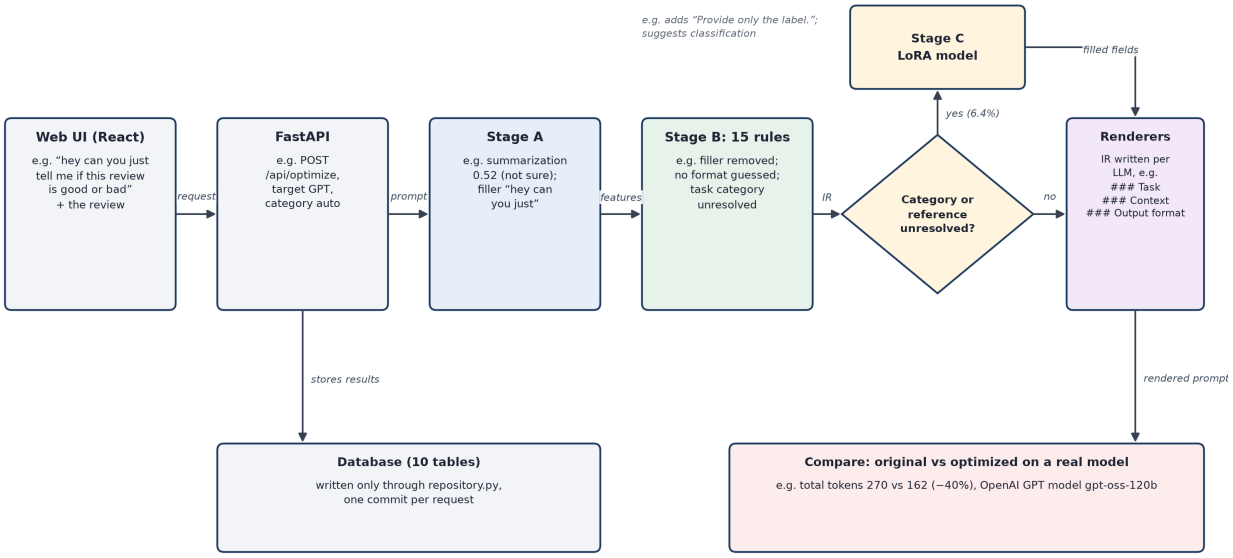


Figure 5.2: *System architecture with the review example under each component*

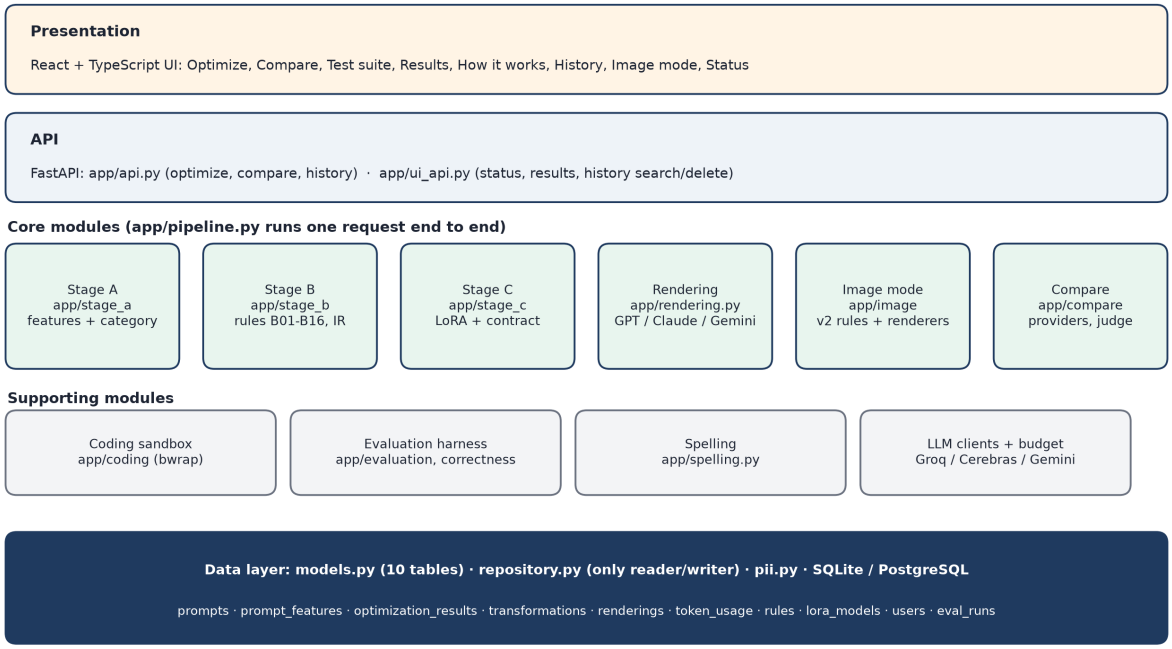


Figure 5.3: *Module diagram: core modules, supporting modules and the data layer*

Table 5.1: Main modules

Module	Responsibility
app/stage_a/	feature detection: classifier, detectors A02-A05, PromptFeatures
app/stage_b/	IR, rules B01-B15 (+ B16 in the app), optimizer, routing
app/stage_c/	training data, parser, training script, runtime, contract and checker
app/rendering.py	renderers for GPT, Gemini and Claude; token counts

Module	Responsibility
app/pipeline.py	runs one request end to end and stores it through the repository
app/api.py, app/ui_api.py	FastAPI endpoints for the user interface
app/compare/	Compare on real models, providers, budgets
app/db/	models (10 tables), repository, PII scrubbing, retention
app/evaluation/, app/correctness/	evaluation harness, judge, token evaluation, correctness suite
app/coding/	sandboxed test runner for coding answers
frontend/	React + TypeScript user interface

5.3 Pipeline flow of one request

One optimization request is handled as one database transaction. The steps below are numbered as in the figure; every arrow in the figure is labelled with what passes along it.

1. The user submits the prompt, the target LLM, the category (auto or one of five) and an optional attachment type (POST /api/optimize).
2. Old data is cleaned: prompts older than 30 days, and everything derived from them, are deleted.
3. The prompt is saved after e-mail addresses and phone numbers are replaced by [EMAIL] and [PHONE] (table prompts).
4. Stage A detects the features (table prompt_features).
5. Stage B, and Stage C if the prompt is routed, produce the IR and the change log (tables optimization_results and transformations, one row per change).
6. The IR is rendered for GPT, Claude and Gemini and the input tokens are counted (tables renderings and token_usage).
7. Decision: did every step succeed? If yes, the request is committed once; if any step failed, the whole request is rolled back and nothing is stored.
8. The response returns the features, the rules that fired, the renderings and the token counts to the user interface.
9. Optionally, the user runs Compare: both prompts go to the same model and the run is stored.
10. Later, History lists, searches, shows and deletes past prompts; deleting a prompt deletes everything derived from it.

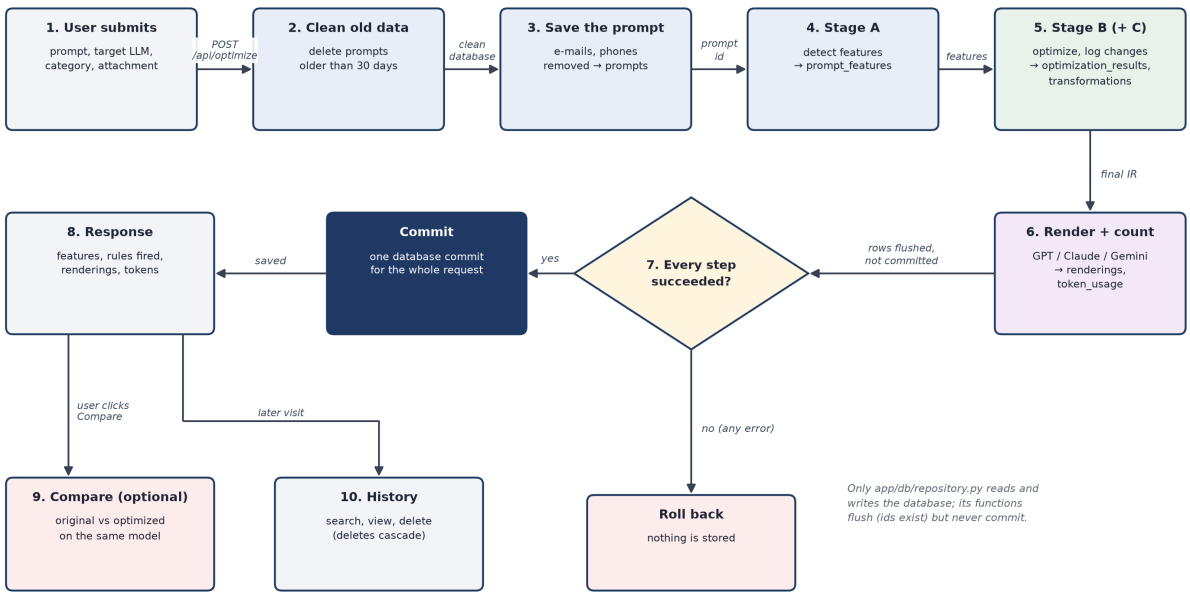


Figure 5.4: *Pipeline flow of one request, with the commit decision at step 7*

Only the repository module (app/db/repository.py) reads or writes the database. Its functions flush, so that the identifiers of new rows exist, but never commit; the endpoint commits once at the end. A failure half-way therefore leaves nothing behind, and the privacy rules are applied in one place.

5.4 Database design

The database has ten tables, defined with the SQLAlchemy 2.0 ORM. The prompt-side tables are deleted in cascade with their prompt, so the retention purge removes everything derived from an expired prompt. The evaluation_runs table is deliberately not linked to prompts, so benchmark results are never purged. Constraints are enforced in the schema: a confidence between 0 and 1, a Stage B transformation must reference a rule, one rendering per result and target.



Figure 5.5: *Entity-relationship diagram (10 tables)*

Table 5.2: Tables of the database

Table	Content	Written by
prompts	prompt text after PII scrubbing, number of redactions, created and expiry time	create_prompt
prompt_features	task type, confidence, missing constraints, filler, ambiguous references	save_features
optimization_results	IR (JSON), optimized text, confidence, whether Stage C was used, latency	save_optimization
transformations	one row per change: step number, stage (B or C), rule, text before and after	save_optimization
renderings	the rendered prompt per target LLM	save_renderings
token_usage	original and optimized input (and output) tokens per target, tokenizer	save_token_usage
rules	the rule catalogue A01-A05, B01-B16 with an enabled flag	seed

Table	Content	Written by
lora_models	metadata of Stage C adapters	register_lora_model
users	optional display name (no e-mail or real name stored)	–
evaluation_runs	one row per evaluated item, variant and target: tokens, latency, quality, task success	record_evaluation

5.5 Evaluation flow

The evaluation runs each split through the variants being compared (the vague prompt, Stage A + B, Stage A + B + C, and the dataset's own optimized prompt) and measures each variant with the metrics of Section 4.10.

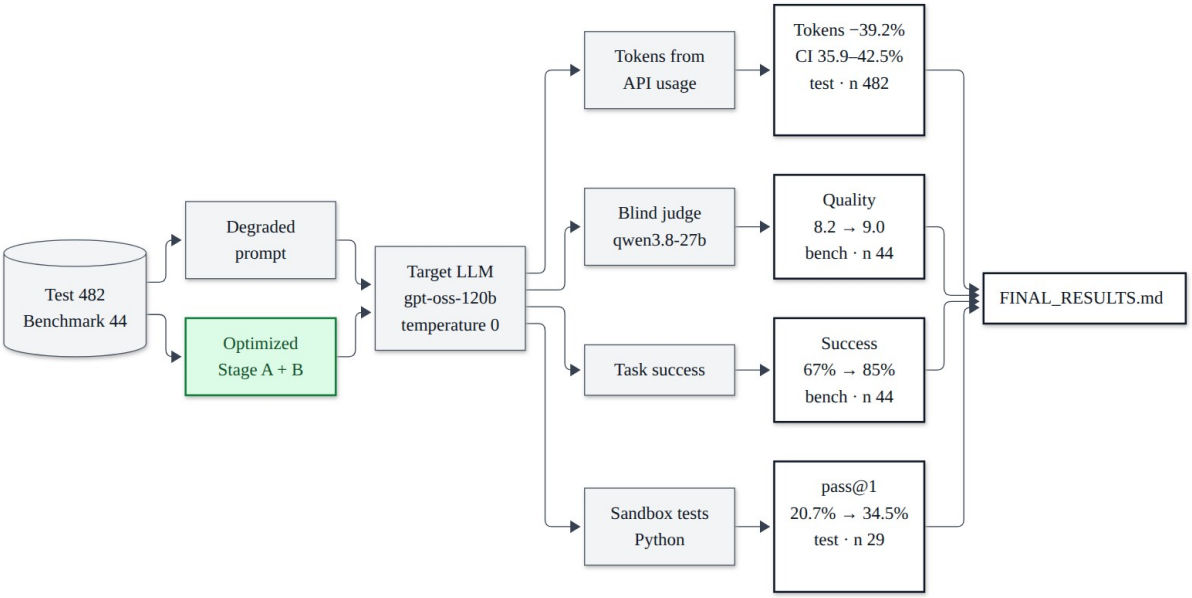


Figure 5.6: *Evaluation flow: splits, variants and metrics*

6. Test plan

Testing is automated and has three levels. Unit tests check every detector, rule and module on its own; integration tests check the modules working together, through the API and through the correctness suite of real cases; system tests run the complete application in a browser and the complete pipeline on the dataset splits. Unit and integration tests run with pytest and need no network: LLM providers are faked or replayed from recorded answers, and database tests run on an in-memory SQLite database (and on PostgreSQL when a test database is configured). Browser tests run with Playwright against the real FastAPI application on a throw-away database. Manual testing covered the demonstration flow and the human review of the dataset and of the correctness-suite gold answers.

Table 6.1: Types of testing and tools

Level	Tool	What is tested	Result
Unit	pytest	every Stage A detector, every Stage B rule, Stage C contract, renderers, database, sandbox, clients	720 test cases, 60 per module, all pass
Integration (API)	pytest + FastAPI TestClient	endpoints end to end, pipeline storage, Compare with recorded answers, retention	included in the 720
Integration (correctness)	correctness suite	50 hand-written cases, 10 per category, with verifiable answers	vague 37/50, optimized 42/50
System (UI)	Playwright + Chrome	simple app and developer view: optimize, token usage analysis, compare, test suite, history, image mode, tour, console errors	16 / 16 passed
System (pipeline)	evaluation harness	482-prompt test split, 44-prompt benchmark, ablation	Chapter 7
Regression	freeze check	Stage A + B on 482 test prompts against the frozen code	byte-identical
Security	sandbox tests	no network, read-only system, memory, time and process limits	passed

6.1 Unit testing

Every Stage B rule is a separate pure function with its own tests, and every Stage A detector has positive and negative tests. Each of the 12 modules has exactly 60 test cases, 720 in all; a check that runs over a whole example set (for instance every example prompt of image mode) counts as one test case. All 720 pass. The 68 database-backed cases run a second time on PostgreSQL, and all 68 pass there too; this is why pytest reports 788 tests (720 + 68). The commands are: cd backend

&& source .venv/bin/activate && python -m pytest -q (788 passed) and python -m app.test_report, which writes the per-module table below.

Table 6.2: Unit test cases per module and their pass rate

Module	Test cases	Passed	Pass rate
Stage A: feature detection	60	60	100%
Stage B: rule-based optimizer	60	60	100%
Stage C: small-model fallback	60	60	100%
Rendering and pipeline	60	60	100%
Image mode	60	60	100%
Database and privacy	60	60	100%
API, web UI, spelling and demo	60	60	100%
Coding tests and sandbox	60	60	100%
Token usage analysis and live models	60	60	100%
Evaluation harness	60	60	100%
Dataset building	60	60	100%
Dataset repair and checks	60	60	100%
Total	720	720	100%

The test-case table below shows four of these cases for each of ten modules, each taken from the real test suite, with the input, the expected and the actual result.

Table 6.3: Unit and module test cases: four sample cases per module

ID	Module	Test	Input	Expected	Actual	Status
TC01	Stage A	Format detected	“Answer in 3 bullet points”	format found (A02)	found	**PASS**
TC02	Stage A	Filler detected	“hey can you please summarize…”	“hey”, “can you please”	as expected	**PASS**
TC03	Stage A	Ambiguous reference only without text	“summarize this” with / without text	flagged / not flagged	as expected	**PASS**
TC04	Stage A	Unlike anything known → other	prompt with no similar train prompt	category = other	other	**PASS**
TC05	Stage B	B01 removes filler	“hey can you please summarize the text for me?”	“Summarize the text.”	as expected	**PASS**
TC06	Stage B	B05 defaults to Python	“Write a function that reverses a list.”	“Use Python.”	as expected	**PASS**
TC07	Stage B	B08 group fallback	“when was it finished” + passage, closed_qa 0.45	“Answer from the provided text in at most three sentences.”	as expected	**PASS**
TC08	Stage B	Running twice adds nothing	an optimized prompt optimized again	no new change	none	**PASS**
TC09	Stage C	Category request	unresolved task category	asks for format, constraints, category	as expected	**PASS**

ID	Module	Test	Input	Expected	Actual	Status
TC10	Stage C	Invalid answer falls back	malformed JSON / wrong keys	Stage B's IR kept	kept	**PASS**
TC11	Stage C	Dangling reference rejected	task “Summarize the provided text”, no text	rejected	rejected	**PASS**
TC12	Stage C	Not routed means not called	nothing unresolved	Stage C not called	not called	**PASS**
TC13	Rendering	No field lost	13 IRs × 3 targets, parsed back	equal to the IR	equal	**PASS**
TC14	Rendering	Claude: XML, context first	IR with context	<context> before <task>	as expected	**PASS**
TC15	Rendering	Fence longer than data	data with ``` inside	fence of 4+ backticks	as expected	**PASS**
TC16	Rendering	Token counts labelled	rendered prompt	GPT exact, Claude/Gemini approx.	as expected	**PASS**
TC17	Database	PII scrubbed before storage	e-mail + two phone numbers	1 [EMAIL], 2 [PHONE], 3 redactions	as expected	**PASS**
TC18	Database	Retention purge cascades	a 31-day-old and a fresh prompt	old prompt and its rows deleted	deleted	**PASS**
TC19	Database	Schema constraints hold	confidence 1.5; Stage B step without rule	rejected by the database	rejected	**PASS**
TC20	Database	Full flow and history	optimize, then history	every stored part returned	returned	**PASS**
TC21	API	Blank prompt	POST /api/optimize, prompt “ ”	HTTP 422 (not 500)	422	**PASS**
TC22	API	Expired prompts hidden	prompt older than 30 days	deleted before history is listed	deleted	**PASS**
TC23	API	Unsure category reported	low-confidence prompt	UI asked to confirm	as expected	**PASS**
TC24	API	Models load once	parallel requests	one Stage A / Stage C instance	one	**PASS**
TC25	Compare	End to end with recorded answers	prompt + model	both answers, tokens, change %	as expected	**PASS**
TC26	Compare	Both prompts PII-scrubbed	prompt with an e-mail	provider sees [EMAIL] only	as expected	**PASS**
TC27	Compare	Budget respected	daily budget used up	refused before calling the model	refused	**PASS**
TC28	Compare	Rate limits	per-minute / per-day 429	retry / stop for the day	as expected	**PASS**
TC29	Sandbox	No network	code opening a socket to 1.1.1.1	connection blocked	blocked	**PASS**
TC30	Sandbox	Time limit	infinite loop	killed at the timeout	killed	**PASS**
TC31	Sandbox	Host files hidden	code reading the home folder	not visible	not visible	**PASS**
TC32	Sandbox	Refuses without bwrap	bubblewrap missing	refusal, code not run	refused	**PASS**
TC33	Image mode	No user word lost	image prompts (parametrized)	every word kept, no default added	as expected	**PASS**
TC34	Image mode	Suggestions only for gaps	prompt that names a style	no style suggestion	as expected	**PASS**

ID	Module	Test	Input	Expected	Actual	Status
TC35	Image mode	Aspect ratio only when implied	prompts with and without an implied shape	ratio only when implied	as expected	**PASS**
TC36	Image mode	Accepted suggestions marked	click two suggestions	added after the user's words	as expected	**PASS**
TC37	UI (Playwright)	Every stage shown	click an example	Stage A, B, C and rendering shown	shown	**PASS**
TC38	UI (Playwright)	Unsure category asks the user	low-confidence prompt	confirmation box shown	shown	**PASS**
TC39	UI (Playwright)	History works	list, search, view, delete	all four work	work	**PASS**
TC40	UI (Playwright)	No console errors	every page	0 console errors	0	**PASS**

Examples of real unit tests (backend/tests):

```
def test_b05_defaults_to_python():
    text = "Write a function that reverses a list."
    assert b.b05_add_language(ir_of(text, "coding"),
                              feats(text, "coding")).constraints == ("Use
Python.",)

def test_pii_is_scrubbed_before_storage(db):
    p = repo.create_prompt(db, "mail rahul.k@example.com or call 98765 43210 /
555-123-4567 about the report")
    db.commit()
    stored = db.scalar(select(Prompt.original_text).where(Prompt.id == p.id))
    assert stored.count("[EMAIL]") == 1 and stored.count("[PHONE]") == 2
    assert p.pii_redactions == 3

def test_blank_prompt_is_rejected_with_422_not_500(client):
    r = client.post("/api/optimize", json={"prompt": "   \n ", "target": "gpt"})
    assert r.status_code == 422 and "empty" in r.json()["detail"]
```

6.2 Integration testing

6.2.1 API integration

API tests send real HTTP requests through the FastAPI test client and check the whole chain: the pipeline with and without Stage C, the storage of every part through the repository, History, Compare with recorded model answers, image mode, input errors (HTTP 422 instead of 500), retention before History is listed, and removal of personal data from what Compare sends to the provider.

6.2.2 Correctness suite, by category

The correctness suite tests the optimizer together with a real model on 50 new, hand-written cases that are close to students' daily use: hostel notices, timetables, placement circulars, bank statements, recipes and small coding tasks. There are exactly ten cases per category. Each case has

one verifiable gold answer, confirmed by the team (50 of 50), and a vague prompt written the way a student would type it. The vague prompt and the optimized prompt are both sent to OpenAI's GPT model gpt-oss-120b; the answer is checked deterministically where possible (numbers, item sets, labels, hidden unit tests run in the sandbox) and with a blind checklist judge for summaries. A case passes when the optimized prompt's answer is correct. Cases marked (Stage C) were routed to Stage C.

Table 6.4: Correctness suite: closed_qa (correct answers: vague 8/10, optimized 10/10)

Case	Scenario	Vague prompt	Optimized prompt	Status
cqa-01	hostel fee notice: total for a late second-year payer	wrong	correct	PASS
cqa-02	timetable with a corrigendum that moves one paper	correct	correct	PASS
cqa-03	T20 scorecard without the total: target for the chasing side	correct	correct	PASS
cqa-04	recipe for 4 scaled to 10 people (Stage C)	correct	correct	PASS
cqa-05	placement notice: one company's CGPA cut-off	correct	correct	PASS
cqa-06	recharge plans: cheapest per day with 2 GB/day	correct	correct	PASS
cqa-07	library overdue fine with two slabs	correct	correct	PASS
cqa-08	bus timetable on a Sunday: earliest arrival	correct	correct	PASS
cqa-09	online deal with coupon and capped bank discount	correct	correct	PASS
cqa-10	attendance: lectures one can still miss and keep 75%	wrong	correct	PASS

Table 6.5: Correctness suite: information_extraction (correct answers: vague 7/10, optimized 10/10)

Case	Scenario	Vague prompt	Optimized prompt	Status
ie-01	drive notice: branches eligible for one company	correct	correct	PASS
ie-02	coordinator's e-mail: the revised deadlines	correct	correct	PASS
ie-03	bug thread: OS versions that crash	correct	correct	PASS
ie-04	result sheet: students with a backlog	correct	correct	PASS
ie-05	recipe: dairy ingredients (Stage C)	correct	correct	PASS
ie-06	PNR status: passengers with confirmed berths	correct	correct	PASS
ie-07	lab timetable: labs of one batch	wrong	correct	PASS
ie-08	bank statement: debits above Rs 5,000	wrong	correct	PASS
ie-09	Django traceback: files in the failing traces (Stage C)	correct	correct	PASS
ie-10	scholarship notice: documents for a renewal	wrong	correct	PASS

Table 6.6: Correctness suite: classification (correct answers: vague 7/10, optimized 8/10)

Case	Scenario	Vague prompt	Optimized prompt	Status
cls-01	pressure-cooker reviews: positive / negative / mixed	correct	correct	PASS
cls-02	monthly expenses into four categories (Stage C)	correct	correct	PASS
cls-03	college-app issue triage with a label guide	correct	correct	PASS
cls-04	mess menu: veg / non-veg under the egg rule (Stage C)	correct	correct	PASS
cls-05	placement offers: Normal / Dream / Super Dream	wrong	correct	PASS
cls-06	SMS inbox: legit / promo / scam	correct	correct	PASS
cls-07	student queries routed to the right office (Stage C)	wrong	wrong	FAIL
cls-08	Big-O of short Python snippets	correct	correct	PASS
cls-09	attendance status per student	correct	wrong	FAIL
cls-10	language of each class-group message (Stage C)	wrong	correct	PASS

Table 6.7: Correctness suite: summarization (correct answers: vague 7/10, optimized 5/10)

Case	Scenario	Vague prompt	Optimized prompt	Status
sum-01	accounts circular with an extended date	correct	correct	PASS
sum-02	bug report for a team lead	correct	correct	PASS
sum-03	six laptop reviews into pros and cons	correct	correct	PASS
sum-04	placement-drive notice for WhatsApp	correct	correct	PASS
sum-05	recipe method for a beginner (Stage C)	wrong	wrong	FAIL
sum-06	industrial-visit message with a postponed date	correct	wrong	FAIL
sum-07	fest committee minutes	wrong	wrong	FAIL
sum-08	municipal water-cut notice	wrong	wrong	FAIL
sum-09	mini-project abstract for a slide	correct	correct	PASS
sum-10	PG agreement summarised for parents	correct	wrong	FAIL

Table 6.8: Correctness suite: coding (correct answers: vague 8/10, optimized 9/10)

Case	Scenario	Vague prompt	Optimized prompt	Status
cod-01	SGPA from (grade, credits) pairs	correct	correct	PASS
cod-02	fix an attendance-percentage function	correct	correct	PASS
cod-03	library fine with slabs, cap and reference books	correct	correct	PASS
cod-04	timetable clash detection	correct	correct	PASS
cod-05	Indian digit grouping (lakh/crore commas)	wrong	correct	PASS
cod-06	PRN format validation	correct	correct	PASS
cod-07	fix a class average that crashes on “AB”	correct	correct	PASS

Case	Scenario	Vague prompt	Optimized prompt	Status
cod-08	trip expense splitter	correct	correct	PASS
cod-09	fix a second-highest-marks function	wrong	correct	PASS
cod-10	recipe quantity scaler with fractions (Stage C)	correct	wrong	FAIL

Table 6.9: Correctness suite: accuracy and tokens per category (OpenAI's GPT model gpt-oss-120b)

Category	Vague correct	Optimized correct	Input tokens (vague / optimized)	Output tokens (vague / optimized)	Input + output (vague / optimized)
closed_qa	8/10	10/10	217 / 244	308 / 166	525 / 410
information_extraction	7/10	10/10	244 / 274	289 / 110	532 / 384
classification	7/10	8/10	251 / 282	320 / 254	571 / 535
summarization	7/10	5/10	257 / 274	155 / 170	412 / 444
coding	8/10	9/10	176 / 210	1104 / 354	1279 / 563
all (50)	37/50 (74%)	42/50 (84%)	229 / 257	435 / 211	664 / 467

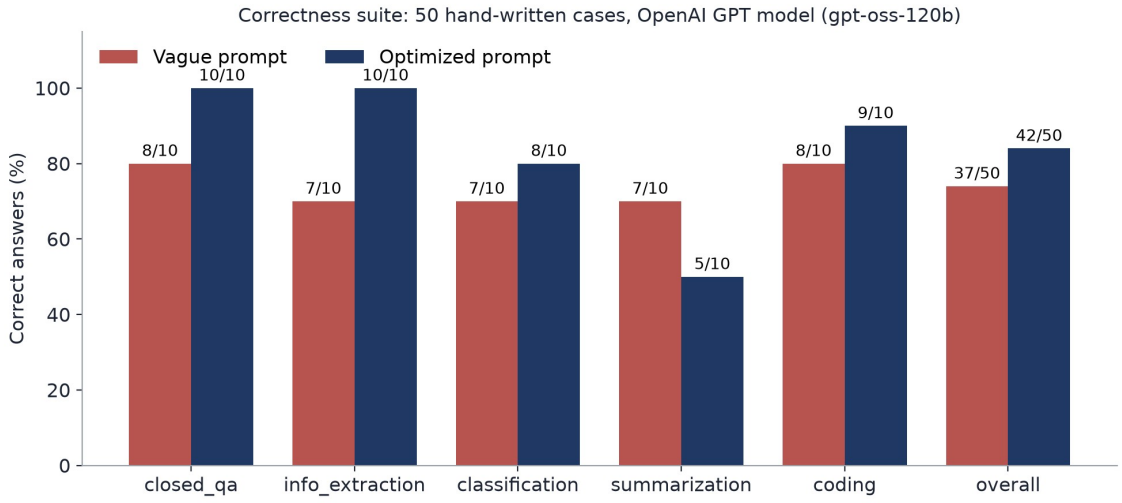


Figure 6.1: *Correctness suite: correct answers per category, vague vs optimized*

Optimization made 9 wrong answers right and 4 right answers wrong; the exact McNemar test gives p = 0.267, so the gain is not statistically significant with 50 cases. On a second provider (Cerebras) the same model gave 43/50 for the vague and 41/50 for the optimized prompts, while total tokens fell by 28.2%: the token saving is robust across providers, the correctness gain is not. The four cases where optimization hurt are analysed in Section 7.4.

6.3 System testing

Table 6.10: Browser end-to-end tests (Playwright, real application)

#	Test	Result
1	example gallery → optimize shows every stage	PASS
2	hovering an issue highlights the words in the prompt	PASS
3	Ctrl+Enter optimizes, undo restores the previous input, live counter updates	PASS
4	possible typos are suggested, and Fix all corrects the prompt and runs again	PASS
5	an unsure category asks the user to confirm	PASS
6	Compare runs both prompts on a real model	PASS
7	test suite: summary, filter and case view	PASS
8	history: listed, searchable, viewable, deletable	PASS
9	theme toggle switches and persists	PASS
10	image mode: suggestions are clickable and per-model renderings	PASS
11	gallery image example opens Image mode and runs it there	PASS
12	tour shows once and can be skipped; ? opens shortcuts	PASS
13	simple app: improve a prompt, then analyse its token usage	PASS
14	simple app: picture prompts, then history shows them and can reuse them	PASS
15	old developer links redirect to /dev	PASS
16	every page loads without console errors	PASS

- Full pipeline on the 482-prompt test split and the 44-prompt benchmark, with a real model and a blind judge (Chapter 7).
- Freeze check: Stage A + B of the current code and the frozen code on all 482 test prompts give byte-identical output (equal SHA-256 hashes).
- Coding sandbox: answers to coding prompts are run against 3-6 assert tests per item, kept only if they pass on the reference solution, in a bubblewrap sandbox with no network and a read-only system.
- Attachment rules: 30 hand-made prompts (5 per attachment type); in all 30 the right rule fires and no other.

7. Results

7.1 Results with screenshots and description

This section first follows the review example through the working application, screen by screen: first the simple app that everyday users see, then every stage in the developer view. It then summarises the results on the full test data.

7.1.1 What the user sees: the simple app

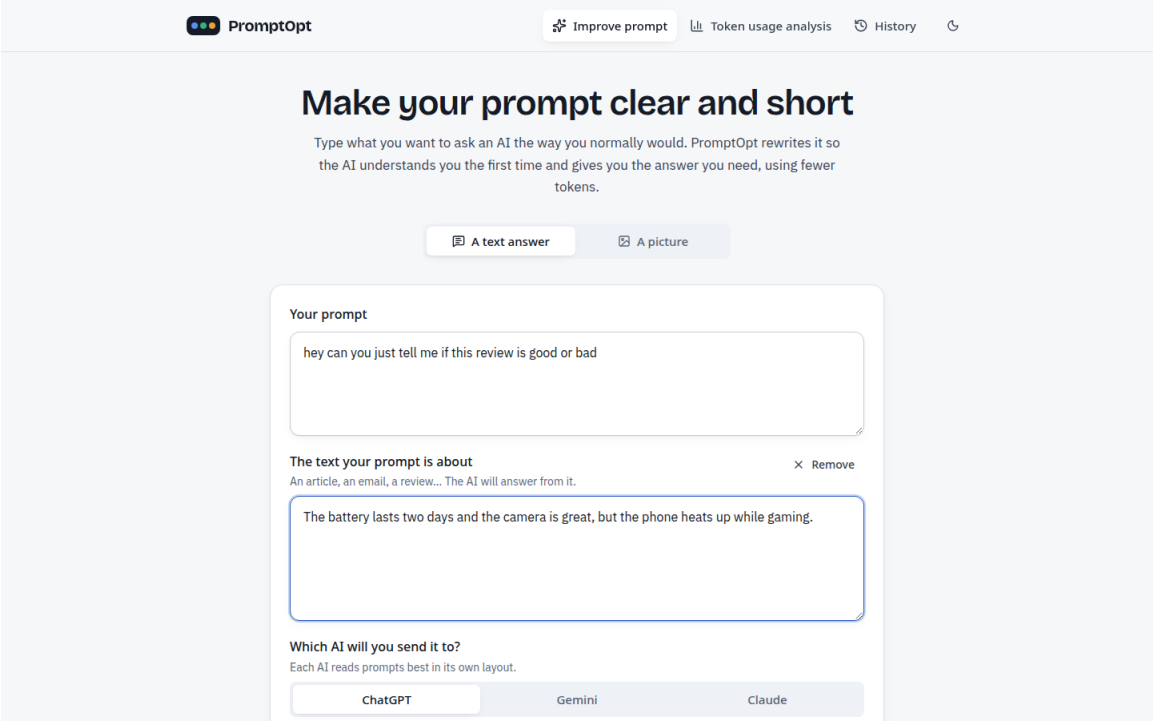


Figure 7.1: *Improve prompt (simple app): the vague prompt, the pasted review and the target AI*

The user types the prompt the way they normally would, adds the review under “The text your prompt is about” and picks the AI they will send it to, here ChatGPT. More options (the kind of task and an attached file) stay folded away; by default the task type is detected automatically.

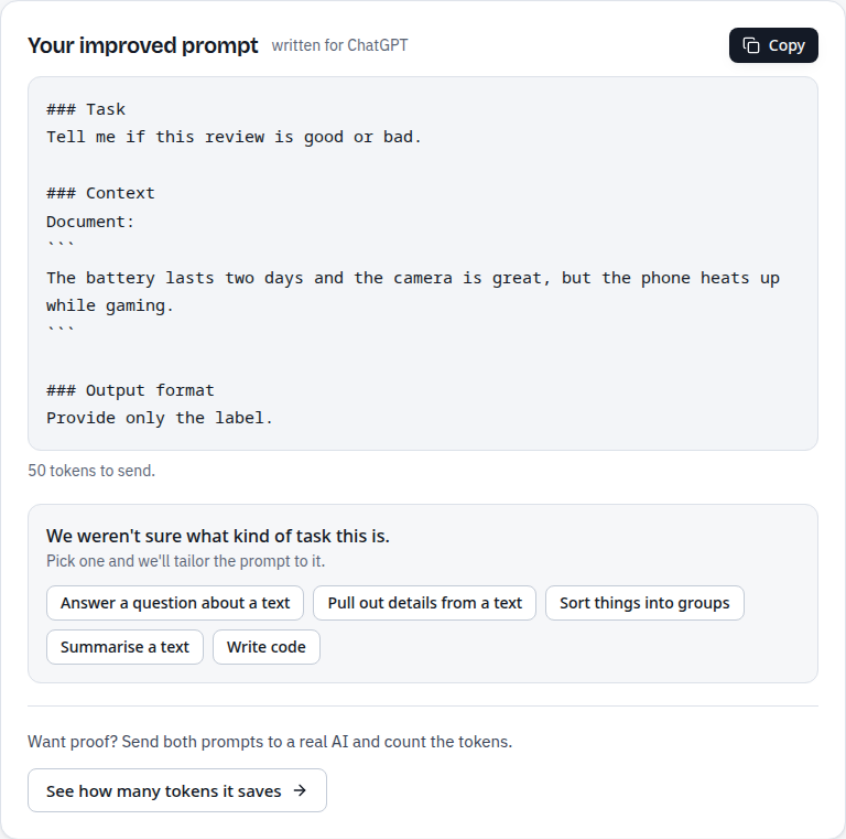


Figure 7.2: *The improved prompt, written in ChatGPT's layout (50 tokens), with the task-type question*

The improved prompt comes back in ChatGPT's layout, with a task, the review as the document and the output format “Provide only the label.”, ready to copy. Because the system was not sure which kind of task this is, it asks the user to pick one instead of guessing; the button below leads to the token usage analysis.

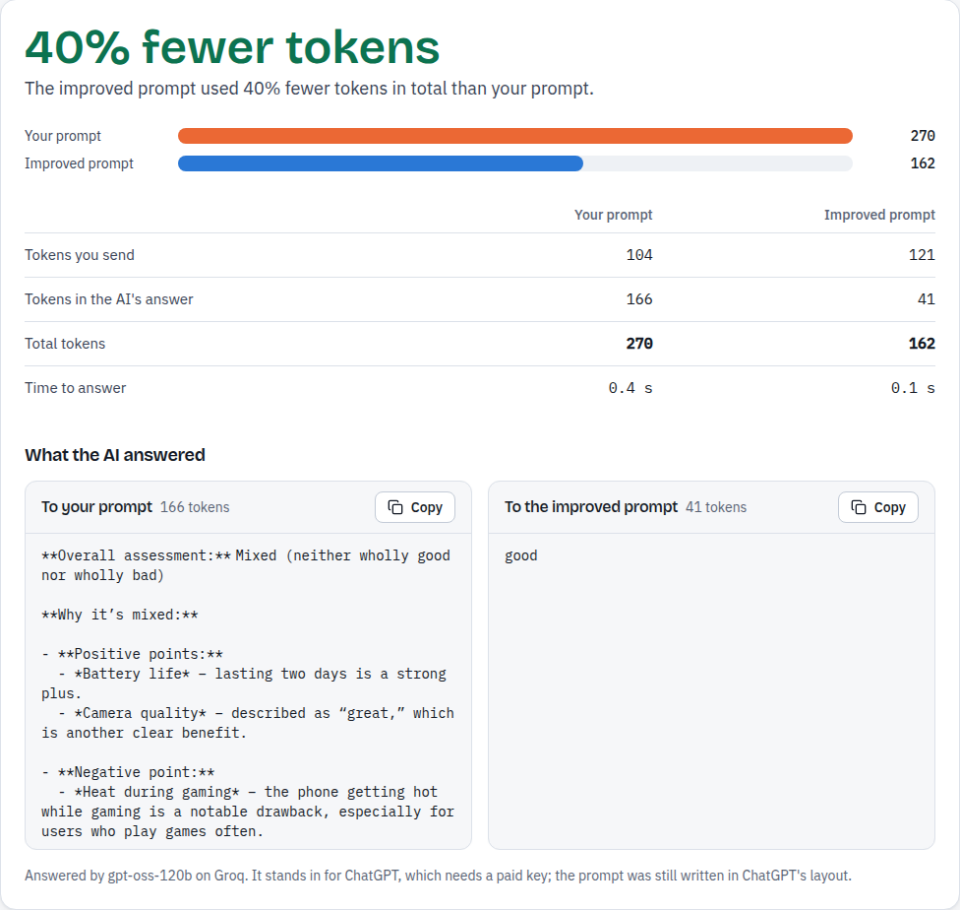


Figure 7.3: *Token usage analysis: both prompts on the same AI, 162 tokens instead of 270 (40% fewer)*

Token usage analysis sends both prompts to the same model, OpenAI's GPT model gpt-oss-120b on Groq, labelled as a stand-in for ChatGPT. The improved prompt is a little longer to send (121 instead of 104 tokens) but the answer shrinks from 166 tokens to 41, the single word "good", so the total falls from 270 to 162 tokens, 40% fewer, and the answer arrives in 0.1 s instead of 0.4 s. The 50 tokens shown on the previous screen are the prompt alone; the model adds about 70 tokens of its own message framing, which is why 121 tokens are sent.

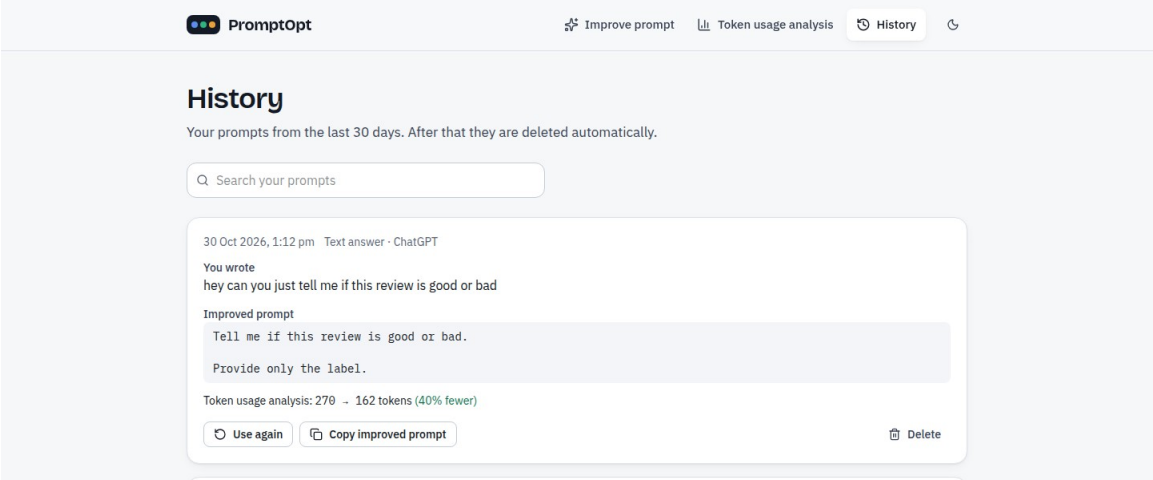


Figure 7.4: *History (simple app): the prompt, the improved prompt and its token saving, kept for 30 days*

History keeps every prompt for 30 days with its improved version and, when it was analysed, the token saving; a prompt can be used again, copied or deleted. E-mail addresses and phone numbers are removed before anything is stored.

7.1.2 Every stage in the developer view

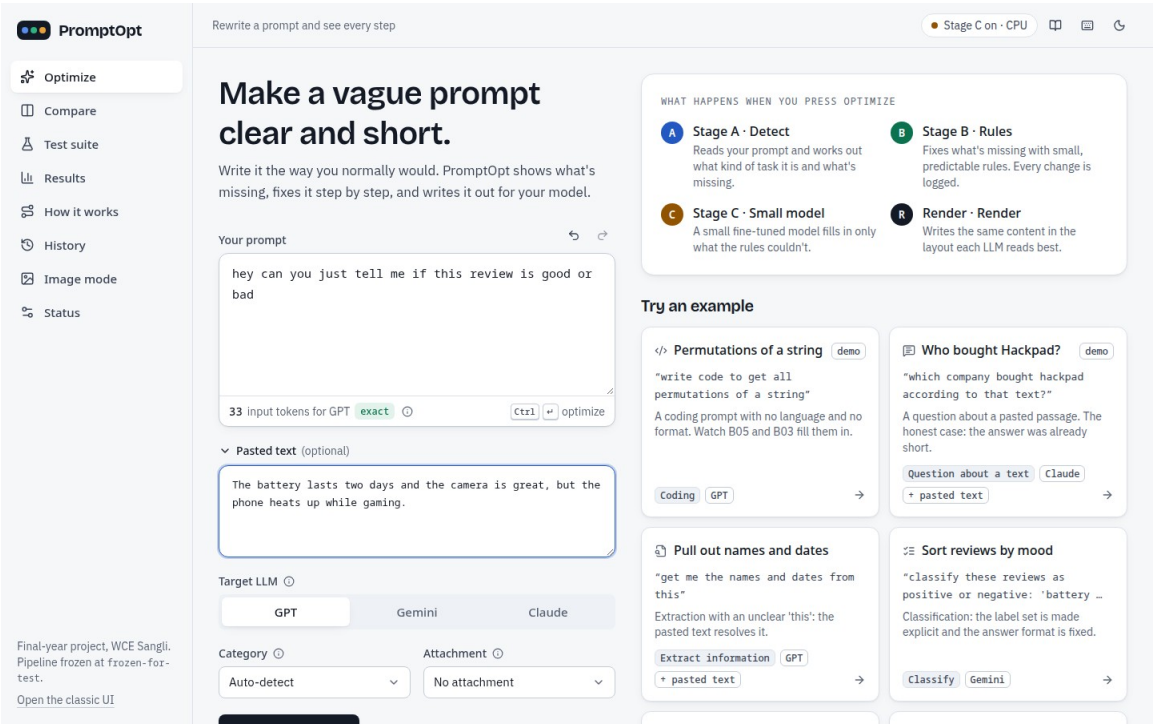


Figure 7.5: *Developer view, Optimize: the vague prompt and the pasted review, target GPT, category auto-detect*

The developer view shows the same request with every stage. The review is pasted under “Pasted text”; the live counter shows 33 input tokens for GPT (counted exactly with OpenAI's tokenizer).

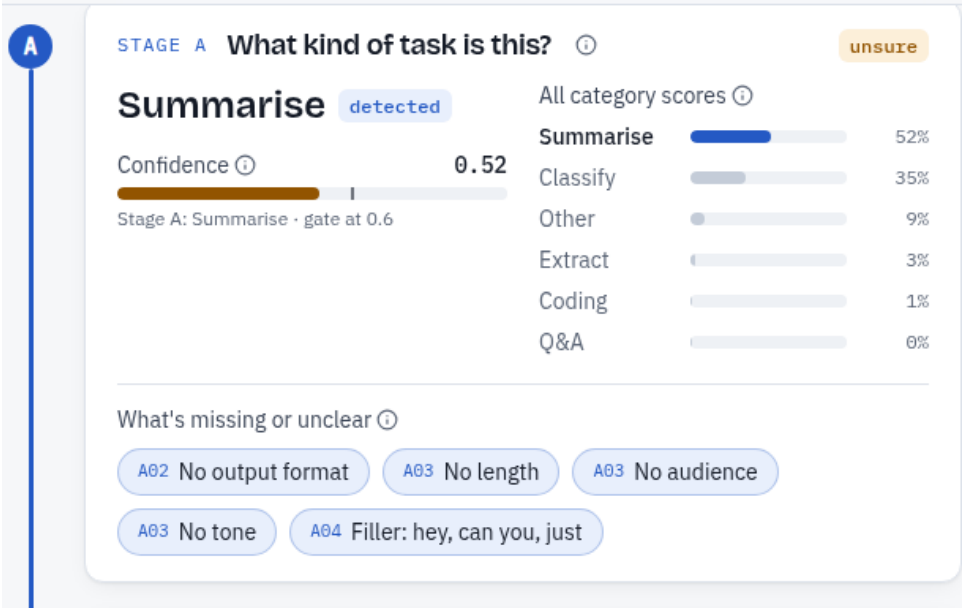


Figure 7.6: *Stage A on the review example: summarization 0.52, below the 0.6 gate, so marked unsure*

Stage A shows its best guess, Summarise, with a confidence of 0.52, and the score of every category (classification 0.35 is second). The confidence bar stops before the 0.6 gate mark, so the category is marked unsure. Below, the detected gaps are listed with their detector codes: no output format (A02), no length, audience or tone (A03), and the filler words “hey, can you, just” (A04).

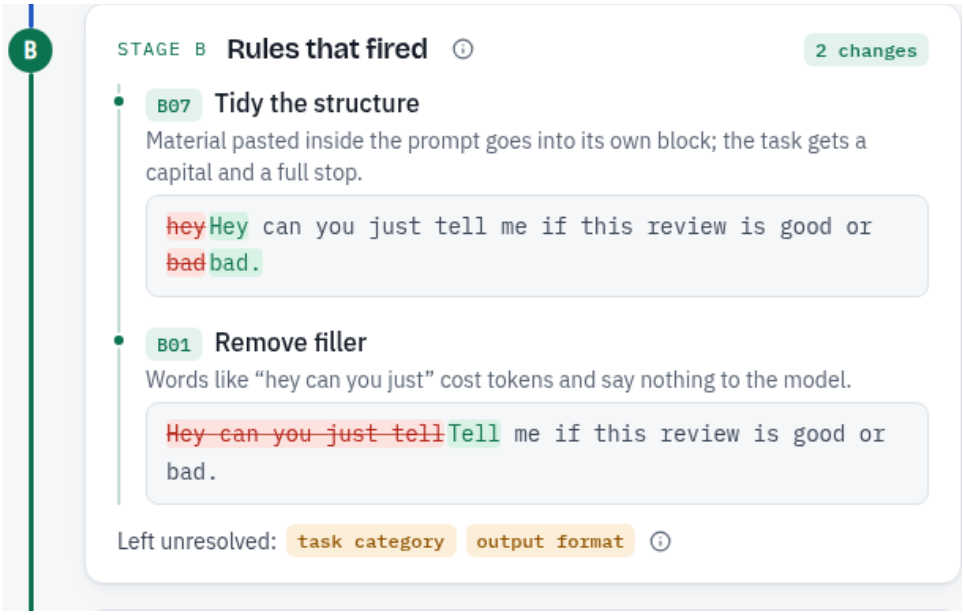


Figure 7.7: *Stage B on the review example: two rules fired, each shown with the text before and after*

Stage B lists the two rules that fired, each with the deleted text in red and the added text in green: B07 tidied the sentence (capital letter and full stop) and B01 removed “Hey can you just”. No

format was added because the category is unsure; the task category and the output format are left unresolved.

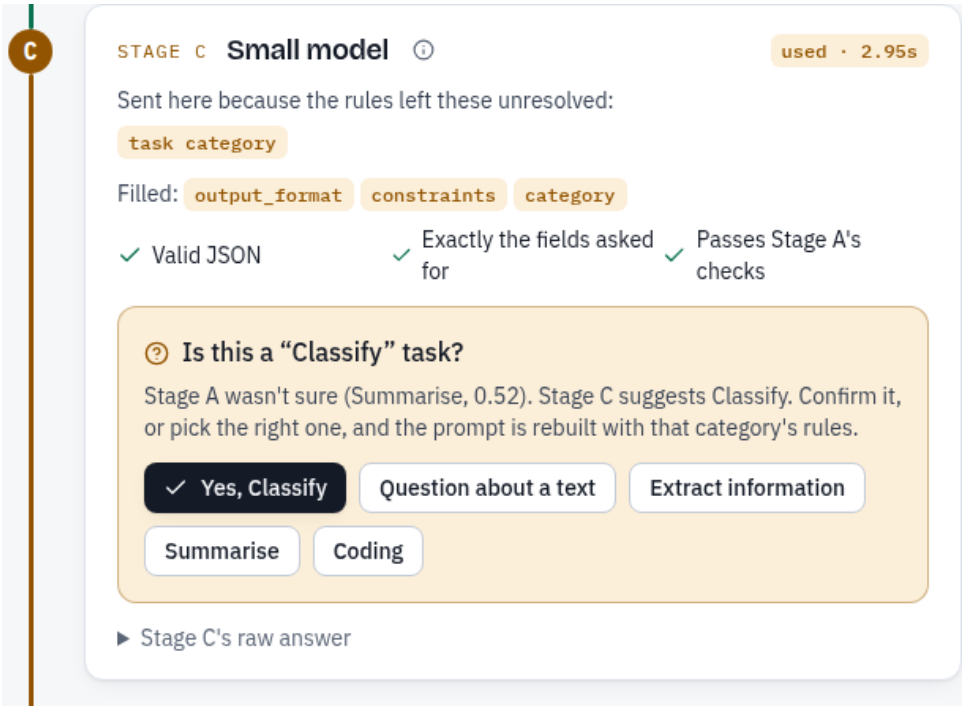


Figure 7.8: *Stage C on the review example: the model filled format, constraints and category; the user is asked to confirm*

Stage C was called because the task category was unresolved. It filled exactly the requested fields, and its answer passed all three checks shown (valid JSON, exactly the requested fields, Stage A's checks). Its category suggestion is not applied automatically: the application asks “Is this a Classify task?” and pre-selects Classify, while the format “Provide only the label.” is applied.

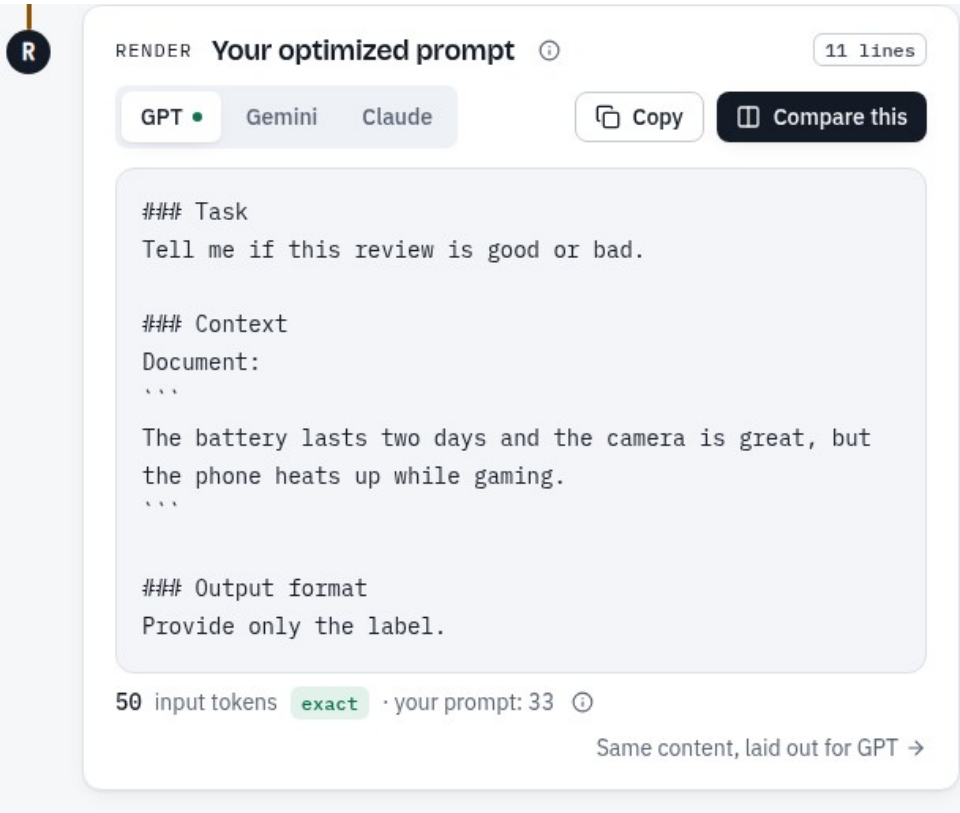


Figure 7.9: *The optimized prompt rendered for GPT, with 50 input tokens*

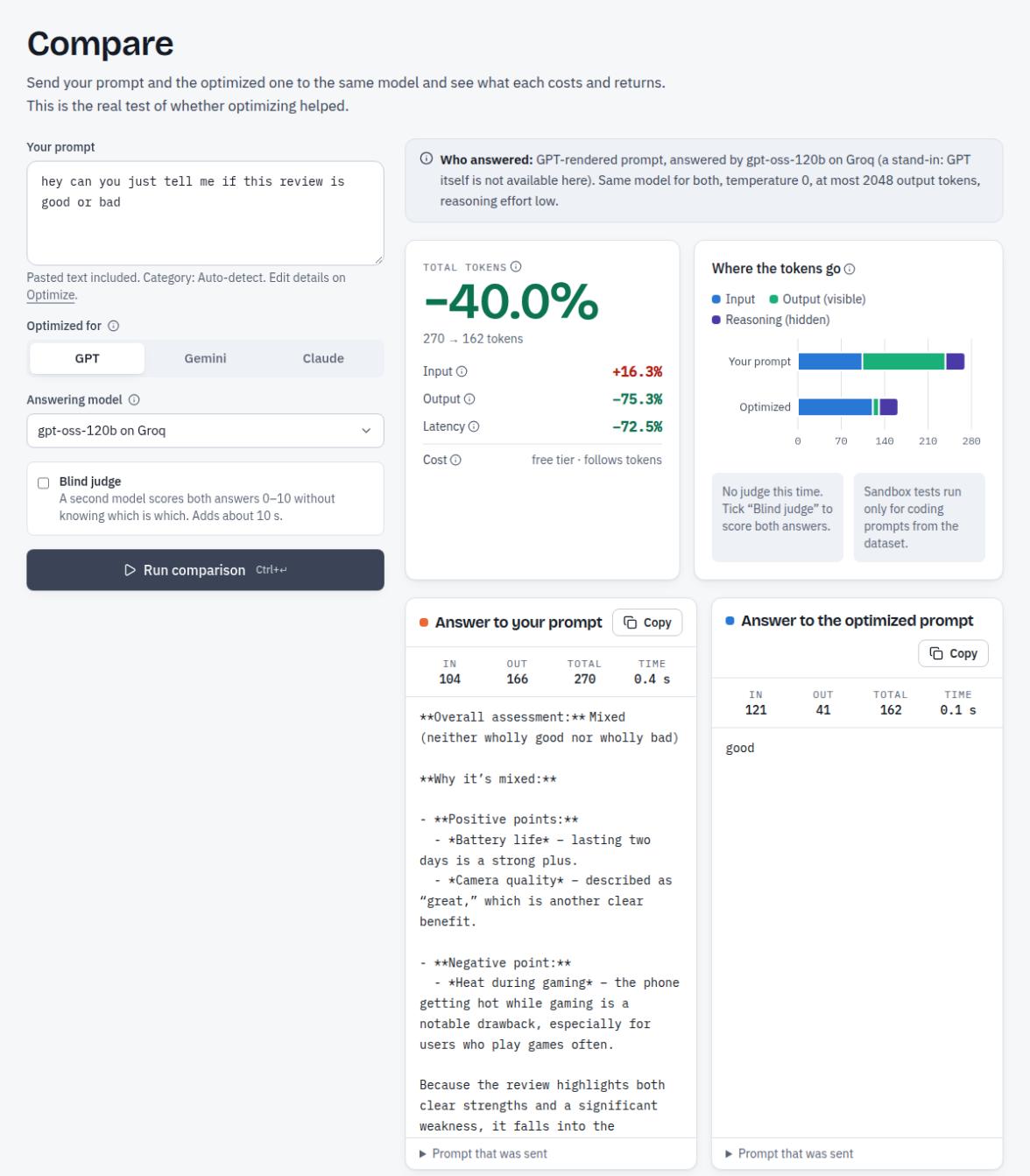


Figure 7.10: *Compare: the vague and the optimized prompt answered by the same model*

Compare sends both prompts to the same model, OpenAI's GPT model gpt-oss-120b on Groq, which stands in for GPT and is labelled as such on the screen. The vague prompt produced a 166-token explanation ("Overall assessment: Mixed …"); the optimized prompt produced the single word "good" in 41 output tokens. Total tokens fell from 270 to 162 (−40.0%) and the response time by 72.5%, even though the optimized prompt is longer (input 104 to 121 tokens, +16.3%).

The vague answer's “mixed” is a fair reading of the review, but it is not one of the two labels the user asked for; the optimized prompt makes the model answer the question as asked.

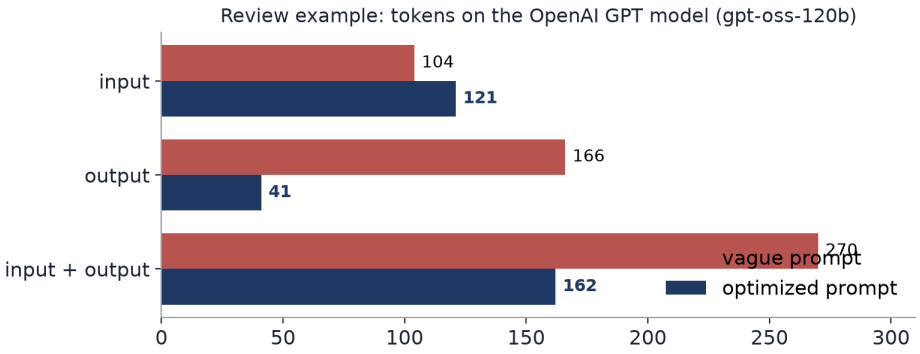


Figure 7.11: *Review example: input, output and total tokens on the same model*

Table 7.1: Headline results on held-out data (output tokens measured on OpenAI's GPT model gpt-oss-120b)

What	Result
Total tokens, optimized vs vague (482 test prompts)	−39.2% per prompt (95% CI 35.9-42.5%)
Output tokens per prompt	165 (vague prompt: 682)
Input tokens per prompt	241 (vague prompt: 226)
Output format stated after Stage B	95.0% (vague prompt: 3.1%)
Stage A category accuracy (vague prompts)	74.7% (macro-F1 0.746)
Prompts routed to Stage C	6.4% (31 of 482)
Benchmark (44 prompts, blind judge): quality / task success	9.0 / 85% (vague prompt: 8.2 / 67%)
Stage C answers passing the checker: base model vs LoRA	5.0% vs 98.2%
Stage C on routed prompts: output format stated	90.3% (without Stage C: 22.6%)
Coding tests pass@1, strict (29 Python items)	34.5% (vague prompt: 20.7%)
Correctness suite (50 cases): vague vs optimized correct	37/50 vs 42/50, total tokens −29.6%
Attachment rules, hand-made set	30/30 correct

7.2 Comparative analysis: input, output and input + output tokens

The table below compares the vague prompt with the optimized prompt (Stage A + B) on all 482 test prompts, separately for input tokens, output tokens and their sum. All counts are the provider's own counts for OpenAI's GPT model gpt-oss-120b; output includes the model's hidden reasoning tokens. Claude and Gemini were not run.

Table 7.2: Mean tokens per prompt: input, output and input + output (482 test prompts, OpenAI's GPT model gpt-oss-120b)

Category	n	Input (vague / optimized)	Output (vague / optimized)	Input + output (vague / optimized)	Change of input + output per prompt [95% CI]
closed_qa	99	279 / 290	835 / 127	1115 / 417	−44.5% [−51.3, −37.6]
information_extraction	93	338 / 351	538 / 115	875 / 466	−24.2% [−32.4, −16.2]
classification	98	98 / 122	390 / 156	488 / 278	−28.5% [−35.8, −21.1]
summarization	94	338 / 348	831 / 204	1169 / 552	−43.3% [−49.4, −37.0]
coding	98	89 / 102	813 / 222	903 / 324	−54.7% [−60.8, −48.0]
all	482	226 / 241	682 / 165	909 / 406	−39.2% [−42.5, −35.9]

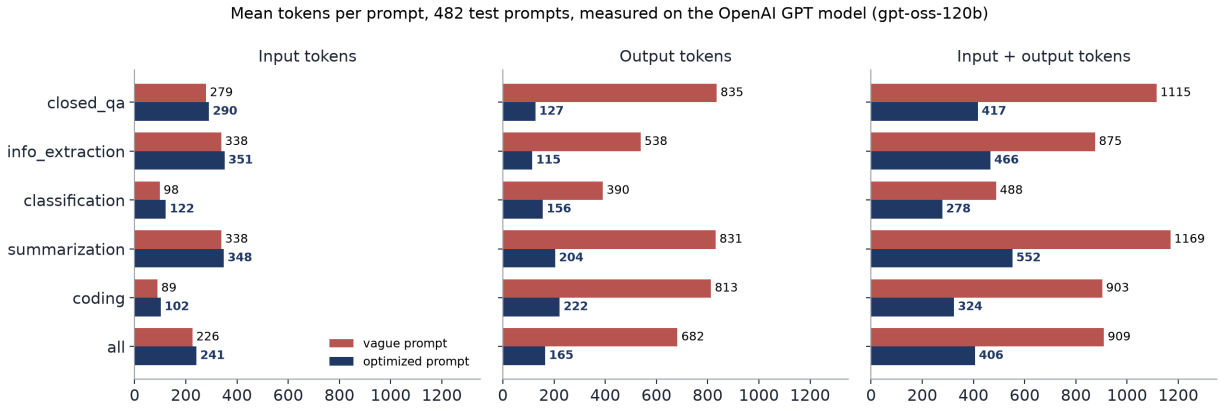


Figure 7.12: *Input, output and input + output tokens per category, vague vs optimized*

Table 7.3: Change per prompt by token type (482 test prompts, OpenAI's GPT model gpt-oss-120b)

Token type	Vague (mean)	Optimized (mean)	Mean change per prompt [95% CI]	Wilcoxon p
input	226	241	+11.1% [10.2, 12.1]	< 0.001
output	682	165	−52.7% [−57.1, −48.2]	< 0.001
input + output	909	406	−39.2% [−42.5, −35.9]	< 0.001

Input tokens grow in every category, by about 4-26% per prompt, because the optimized prompt states a format, a length or labels. Output tokens fall by much more, because the model no longer writes alternatives, explanations and examples nobody asked for. The sum therefore falls in every category, most for coding (−54.7%), where vague prompts produced several versions of the code with explanations. 87 of the 482 prompts (18.0%) individually cost more, mostly short-answer information_extraction prompts (35 of 93) whose answer was already short.

The −39.2% is the mean of the per-prompt changes, so every prompt counts equally. The mean totals in the table (909 and 406 tokens) correspond to the aggregate reduction of 55.4% (Section

4.10), which is larger because a few very long vague answers dominate the totals. This report always quotes the smaller, per-prompt figure.

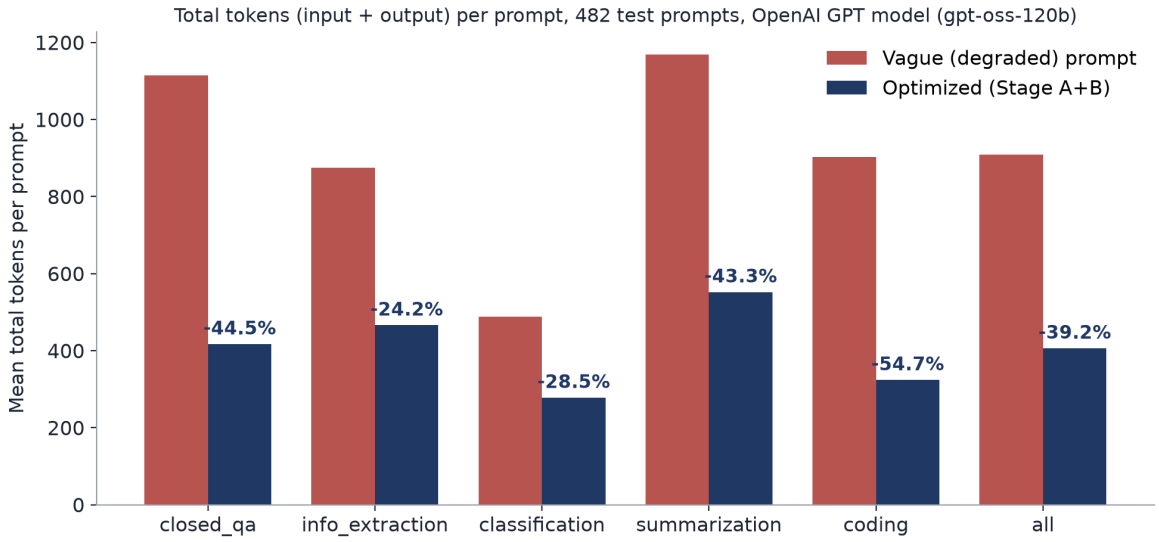


Figure 7.13: *Mean input + output tokens per prompt by category, vague vs optimized*

Table 7.4: Stage C ablation on the test split

Prompt set	System	n	Task intent	Format stated	Fallback
routed	A + B	31	0.869	22.6%	–
routed	A + B + C	31	0.843	90.3%	3.2%
routed	C only	31	0.712	90.3%	6.5%
forced (all)	A + B	482	0.886	95.0%	–
forced (all)	A + B + C	482	0.769	90.2%	2.1%
forced (all)	C only	482	0.765	88.6%	1.9%

Where Stage B routes a prompt, Stage C states a format in 90% of cases instead of 23%, at a small loss in task intent (the similarity of the final task to the original instruction). Sending every prompt through Stage C hurts (task intent falls from 0.886 to 0.769), and Stage C alone without the rules is the weakest, so the rules are worth keeping in front of the model and Stage C stays a fallback for routed prompts only. On the 31 routed prompts, adding Stage C to A + B reduced input + output tokens by a further 26.7% per prompt.

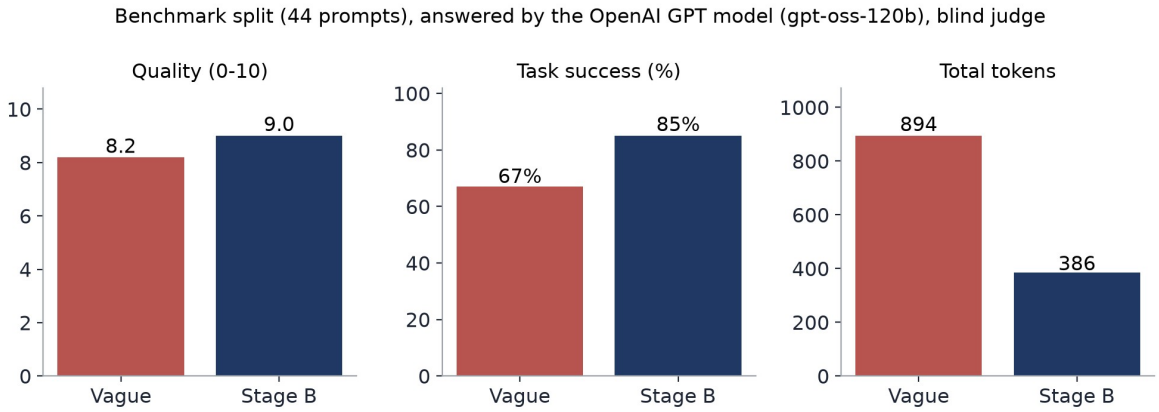


Figure 7.14: *Benchmark split: quality, task success and input + output tokens*

7.3 Performance measures in tabular form and graphs

Table 7.5: Stage A per-category results (482 test prompts)

Category	n	Precision	Recall	F1
closed_qa	99	56.7%	76.8%	0.65
information_extraction	93	73.2%	55.9%	0.63
classification	98	93.1%	95.9%	0.94
summarization	94	60.0%	44.7%	0.51
coding	98	99.0%	98.0%	0.98
accuracy (all)	482			74.7%

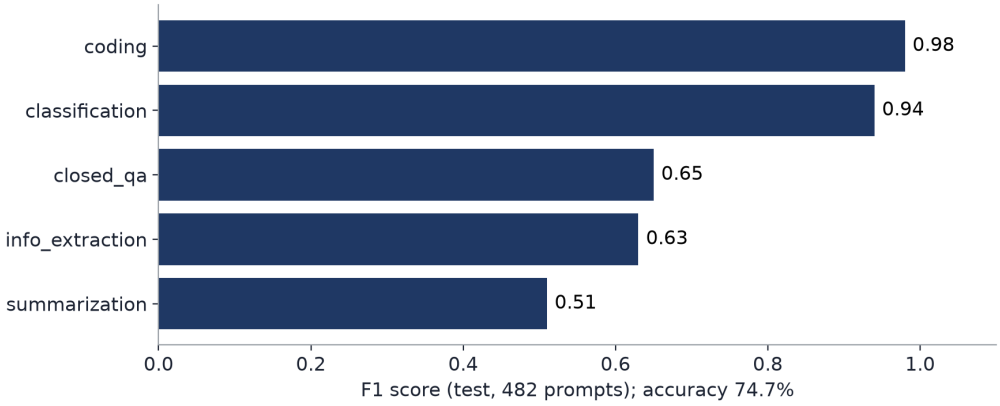


Figure 7.15: *Stage A F1 per category (test split)*

Table 7.6: Stage B per category (482 test prompts)

Category	n	Format stated (vague / Stage B)	Words (vague / Stage B)	Sent to Stage C
closed_qa	99	0.0% / 96.0%	9.1 / 18.8	4.0%
information_extraction	93	4.3% / 94.6%	9.1 / 20.9	5.4%

Category	n	Format stated (vague / Stage B)	Words (vague / Stage B)	Sent to Stage C
classification	98	0.0% / 90.8%	18.7 / 32.5	15.3%
summarization	94	2.1% / 94.7%	8.0 / 16.6	5.3%
coding	98	9.2% / 99.0%	10.6 / 20.8	2.0%
all	482	3.1% / 95.0%	11.2 / 22.0	6.4%

Table 7.7: Per-rule agreement with the dataset's optimized prompts (test)

Rule	Precision	Recall	F1
B01 remove filler	1.000	1.000	1.000
B03 output format	0.824	0.662	0.734
B04 length	0.827	0.315	0.456
B05 language	0.925	0.949	0.937
B06 labels	0.578	0.649	0.612
B07 data moved	0.298	0.824	0.438
B08 group fallback	0.732	0.524	0.611

The per-rule table measures agreement with the LLM-written targets of the dataset, not with a human gold standard: a reasonable addition that the LLM left out counts against the rule. Low recall for B04 and B08 is largely by design, because lengths are added only above the confidence gate and only where they matter.

Table 7.8: Stage C: zero-shot base model vs LoRA (964 test examples)

Model	Valid JSON	Exact keys	Passes the checker	Task similarity
Qwen2.5-0.5B, zero-shot	93.5%	51.2%	5.0%	0.668
Qwen2.5-0.5B + LoRA	100%	100%	98.2%	0.801

Table 7.9: Other components

Component	Measure	Result
Coding (29 Python test items)	pass@1 strict / lenient	strict 34.5% (vague 20.7%); lenient 41.4% (vague 37.9%)
Attachment rules (30 hand-made prompts)	right rule only, in every rendering	30/30
Image mode (30 held-out prompts, Stable Diffusion 1.5)	CLIP similarity to the request	original 33.08, v1 30.78 (worse, p = 0.043), v2 32.77

7.4 Result analysis

Where the saving comes from. The optimized prompt is slightly longer, but the answer is much shorter. The review example shows the mechanism on one prompt (166 output tokens fall to 41), and the test split shows it on 482 prompts (682 output tokens fall to 165). A stated format and length stop the model from writing long, unrequested answers. For a model that prices output higher than input, the money saving is larger than the token saving.

When it costs more. Prompts whose answer is already short can cost more: the added format line costs input tokens without shortening the answer. In Compare, a one-line closed_qa answer cost 17.2% more. Of the 482 test prompts, 87 cost more, mostly short extractions.

Correctness. Optimization helped most where the vague prompt left the expected answer shape open: all ten extraction cases and all ten closed_qa cases were correct with the optimized prompt. The suite viewer of the application shows each case with its gold answer and both answers side by side.

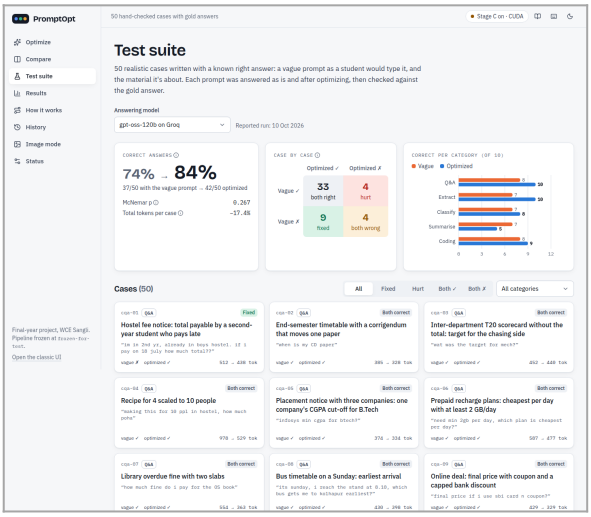


Figure 7.16: *Test suite: correctness summary and the case grid*

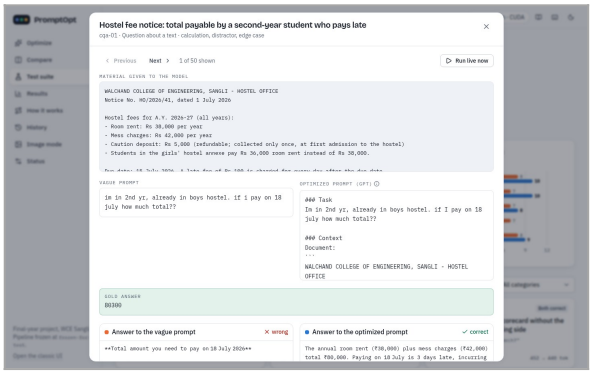


Figure 7.17: *One case (cqa-01): gold answer 80300; the vague answer was wrong, the optimized right*

Honest failures. The four cases where optimization made a correct answer wrong each have a clear cause:

- cls-09: Stage A read an attendance table as code (coding, 0.84, above the gate), so B05 and B03 asked for code; the model returned a Python script instead of a label per student. Choosing the category in the application avoids it.

- sum-06: B03 turned “4-5 lines” into “Use bullet points”; the bulleted answer had every key fact but 92 words against a 90-word limit.
- sum-10: only B07 fired, so the request was the same as the vague one; the answer added a title line and reached 83 words against 80. A near miss, not a rule effect.
- cod-10: routed to Stage C, which added only a code-block format; the model's fraction handling was wrong in 4 of 8 hidden tests although the specification was identical in both prompts.

Stage A is the main weak spot. Closed_qa, information_extraction and summarization are hard to tell apart on vague prompts (F1 0.51-0.65), partly because Dolly's labels in this group are noisy. The 0.6 gate, the group rule B08 and the user's confirmation keep these errors from turning into wrong formats. The Results page of the application collects these numbers from the final results file.

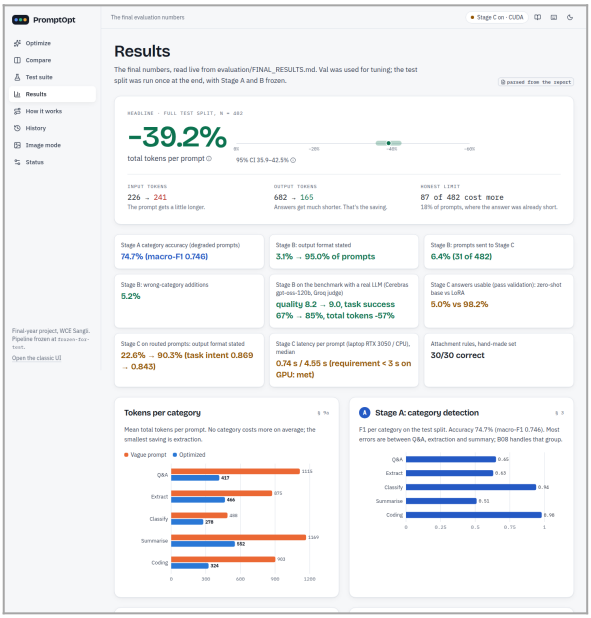


Figure 7.18: *Results page: headline numbers, read from the final results file*

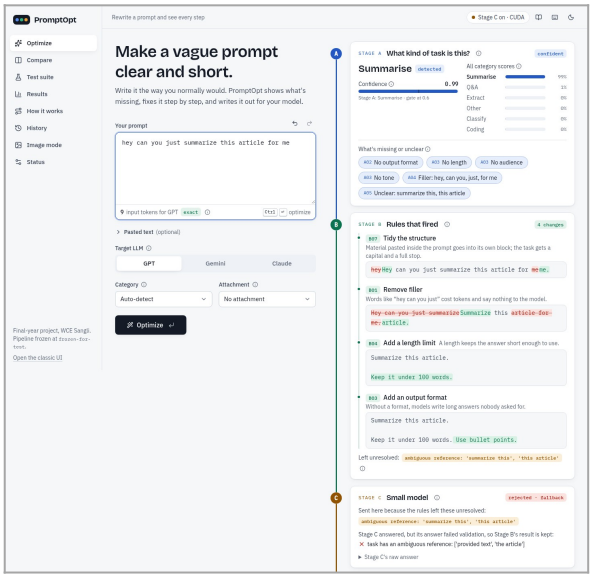


Figure 7.19: *A rejected Stage C answer: the checker failed, so Stage B's result is kept*

Image mode. Version 1 inserted default keywords for every missing attribute (for example “natural lighting”), which pulled Stable Diffusion images away from the request: CLIP similarity fell from 33.08 to 30.78 on 30 held-out prompts ($p = 0.043$). Version 2 keeps the user's words first and offers missing attributes only as clickable suggestions; it shows no measurable harm (32.77) and keeps the requested style in 12 of 12 styled prompts.

7.5 GUI images and screenshots

The application runs as a FastAPI server that also serves the React user interface. The screens below are the ones not already shown in Section 7.1.

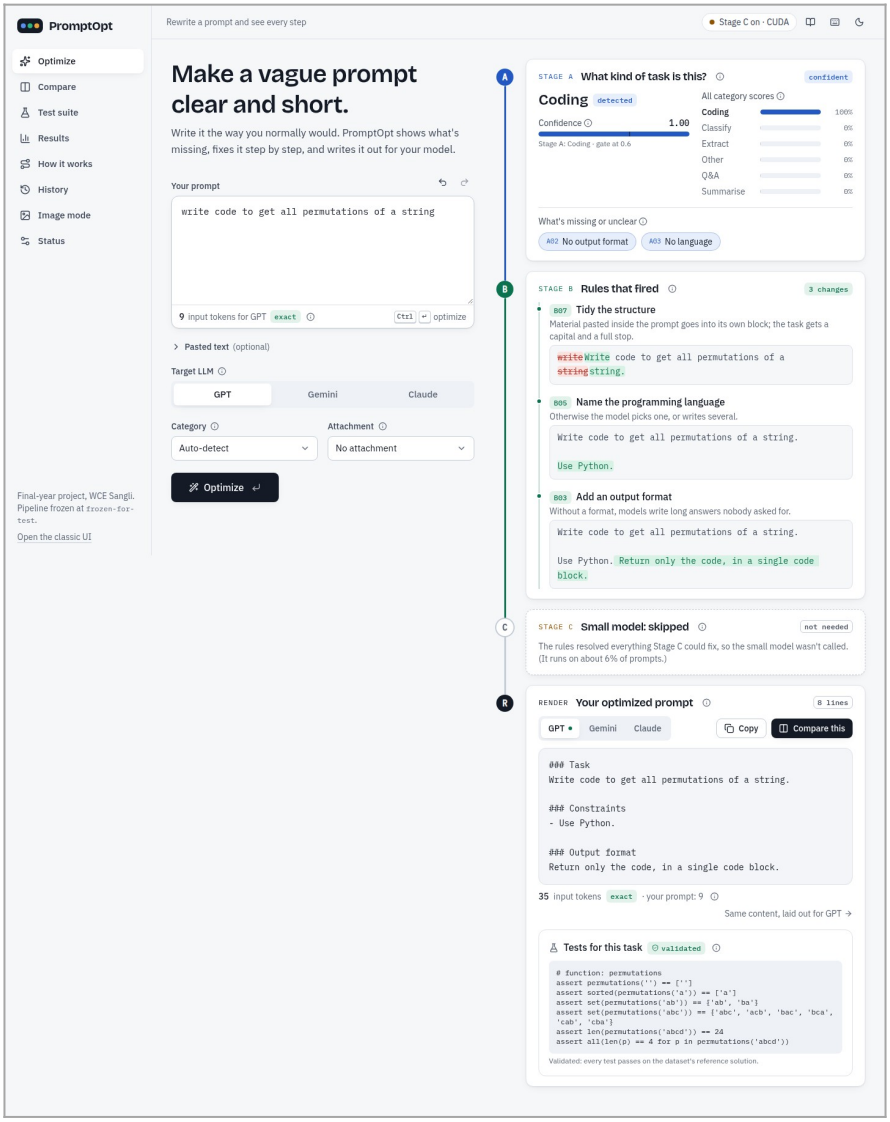


Figure 7.20: *Optimize with a coding example: B07, B05 and B03 fire; Stage C is skipped*

Optimize with the coding example “write code to get all permutations of a string”: Stage A is sure (coding, 1.00), Stage B adds “Use Python.” and “Return only the code, in a single code block.”, and Stage C is skipped because nothing is unresolved.

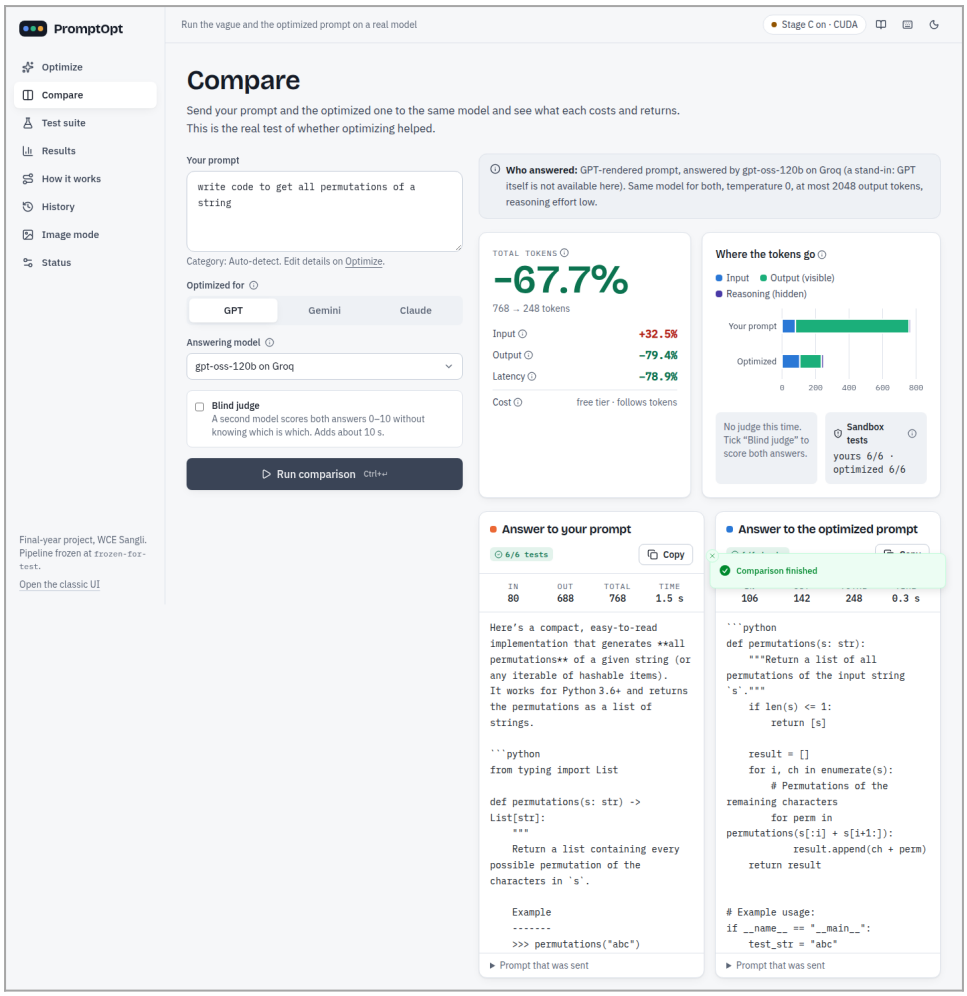


Figure 7.21: *Compare on the coding example: 248 tokens instead of 768 (−67.7%), both answers pass 6 of 6 tests*

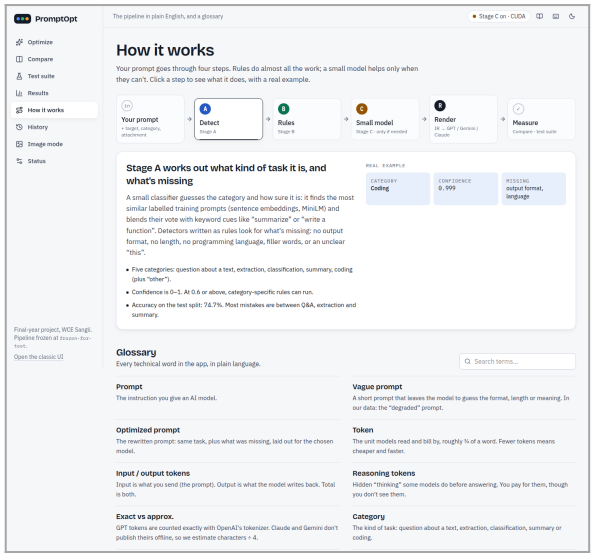


Figure 7.22: *How it works: the pipeline explained inside the application*

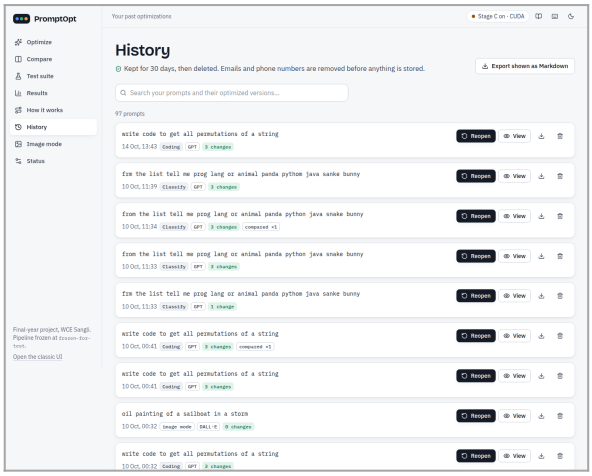


Figure 7.23: *History: search, view and delete; prompts expire after 30 days*

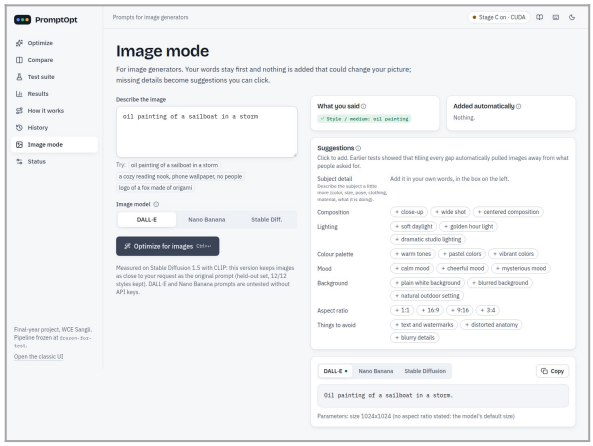


Figure 7.24: *Image mode: the user's words are kept, missing attributes are suggestions*

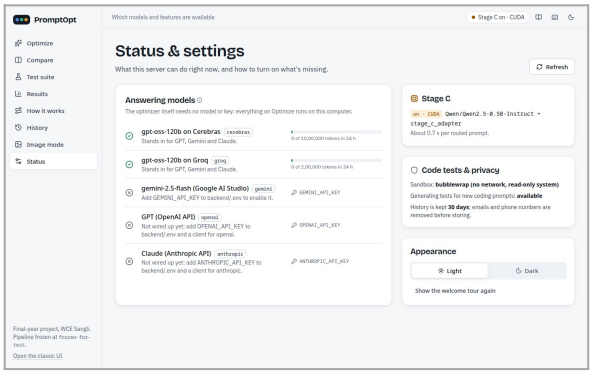


Figure 7.25: *Status: providers with 24-hour usage, Stage C device, sandbox*

7.6 Screenshots of the GitHub project upload

The code is public at https://github.com/piush365/PromptOpt.

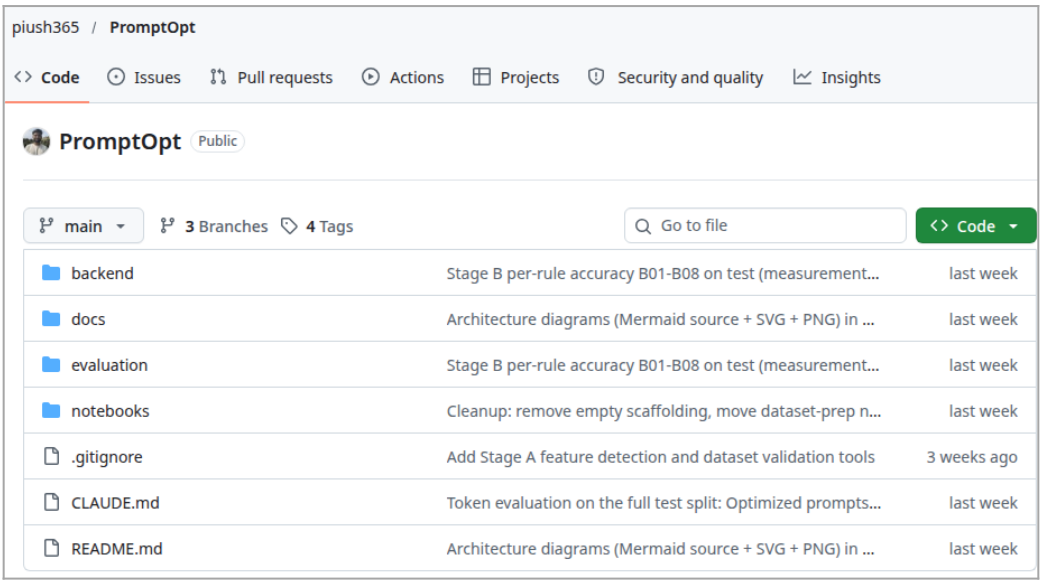


Figure 7.26: *GitHub repository piush365/PromptOpt: branches, tags and top-level folders*

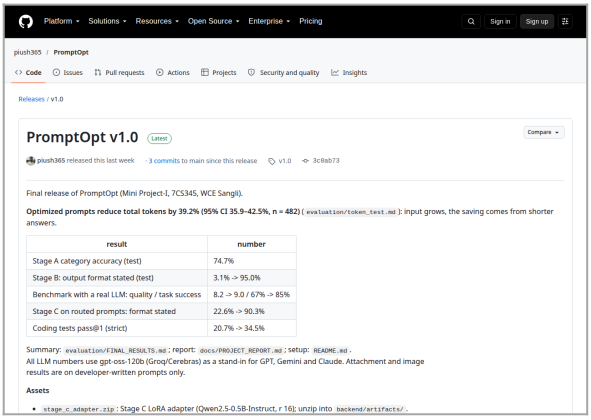


Figure 7.27: *Release v1.0 with the Stage C adapter and the Stage A index*

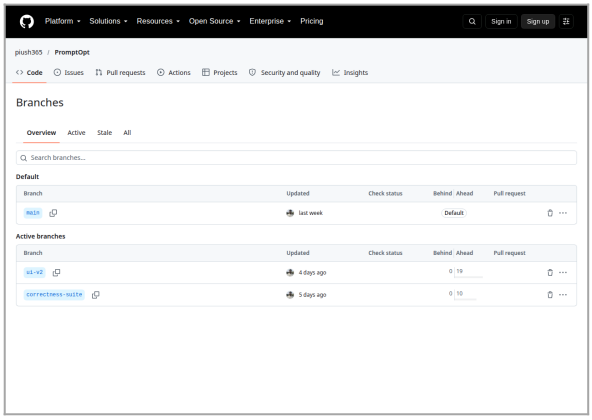


Figure 7.28: *Branches: main (default), ui-v2, correctness-suite*

7.7 Use of Git and GitHub

Table 7.10: Git and GitHub usage

Item	Value
Repository	github.com/piush365/PromptOpt (public)
Commits (all branches)	102 (main 99; ui-v2 3 ahead of main with the final report and deck updates; correctness-suite merged into main)
Branches	main (default), ui-v2 (React user interface), correctness-suite
Tags	frozen-for-test (code for the final numbers), final-for-test (test split run once), v1.0 (release), pre-ui-v2
Release	PromptOpt v1.0: Stage C adapter, Stage A category index, Stage C data manifest

Item	Value
Work tracking	roadmap and phase tables in the repository, tags as evaluation checkpoints, one release; GitHub Issues and Projects were not used

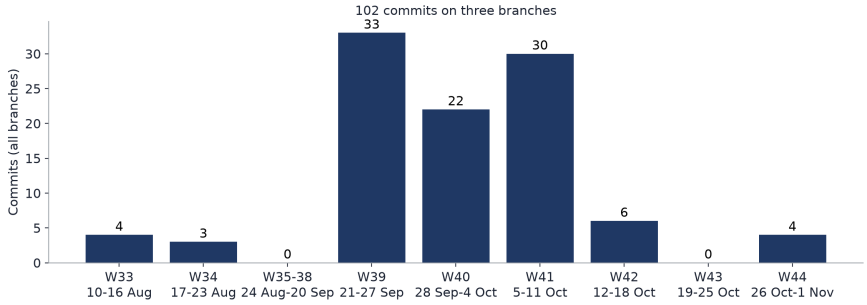


Figure 7.29: *Commits per week (all branches)*

Tags were used as evaluation checkpoints: frozen-for-test marks the code that produced the final test numbers; after it, no Stage A or Stage B rule, threshold or judge was changed, and the freeze check proves it.

7.8 Challenges faced and how we solved them

Table 7.11: Challenges and solutions

Challenge	What happened	How we solved it
Dataset quality	LLM-written optimized prompts dropped data from the instruction or changed the task	automatic checks, deterministic repair with a log, human rating and an LLM-assisted filter
Kappa paradox	raters agreed on 89-94% of rows, but Fleiss' kappa was near 0 because 96-98% of answers were yes	reported Gwet's AC1 (0.92-0.96) next to kappa and explained the paradox
Free API quotas	Groq 200,000 tokens per day per model; Cerebras 150 requests per hour	rolling 24-hour usage ledger, pacing, a 70% budget rule, resumable runs
Stage C hallucination	the zero-shot model passed the checks in only 5% of cases and could keep “this”	LoRA fine-tuning (98.2%), a strict JSON contract, the checker and a fallback to Stage B
Image mode v1	default keywords turned a watercolor request into a photo	version 2: the user's words first, defaults only as suggestions
Regex edge cases	“thank you so much” inside a button text; “tell me X or Y” labels	a unit test per edge case; B16 wider label detection in the application
4 GB GPU	fine-tuning and serving on a laptop GPU	micro-batch 1 × 16, bf16, gradient checkpointing; models loaded once

8. Conclusion

PromptOpt shows that a cheap, explainable, rule-first optimizer can make vague prompts clearer and cheaper. The review example summarises the whole system: Stage A notices that the task type is uncertain and that no answer format is given, Stage B removes the filler without guessing, Stage C supplies the missing format and asks the user to confirm the category, and the same model then answers in 162 instead of 270 tokens. On 482 held-out test prompts, measured on OpenAI's GPT model gpt-oss-120b, optimized prompts reduce input + output tokens by 39.2% per prompt (95% CI 35.9-42.5%): input grows from 226 to 241 tokens, while output falls from 682 to 165. The output format is stated in 95.0% of prompts instead of 3.1%, and on the benchmark quality rises from 8.2 to 9.0 and task success from 67% to 85%. A small LoRA model, trained on a 4 GB laptop GPU, helps exactly where the rules stop (format stated on 90.3% of routed prompts instead of 22.6%) and is kept out of everything else. The working application (FastAPI + React: a simple app with token usage analysis and history, a developer view with every stage, and a ten-table database with one commit per request) is covered by 720 unit test cases (60 for each of 12 modules) and 16 browser tests, all passing.

Limitations of the proposed solution

- All output-token, quality and correctness numbers come from OpenAI's GPT model gpt-oss-120b on free tiers; Claude and Gemini were not run, and their input-token counts are approximate.
- Input tokens grow in every category; 87 of 482 test prompts cost more in total because their answer was already short.
- Stage A reaches 74.7% category accuracy; closed_qa, information_extraction and summarization remain hard to separate.
- Correctness suite: 42/50 vs 37/50 is not statistically significant ($p = 0.267$) and was not reproduced on a second provider (41 vs 43).
- Stage C is trained on LLM-written targets, and the routed test set is small (31 prompts).
- Attachment and image results are on developer-written prompts only; coding tests cover Python only (29 items).
- The kappa target for dataset validation was not met (the kappa paradox), although raw

agreement and AC1 are high.

Future scope and possible extensions

- Lean mode for short-answer prompts: add only the format line, or nothing, when the expected answer is short.
- Send label-set and typo-heavy prompts to Stage C; spelling normalisation that the user confirms.
- Blind user tests with prompts written by people outside the team (the attachment and image test sets are ready).
- Compare and the full evaluation on GPT, Claude and Gemini through their paid APIs.
- A larger correctness suite, with more cases per category and more providers, to reach statistical significance.
- Deployment beyond the local network (PostgreSQL, HTTPS, login); coding tests beyond Python.

9. References

[1] K. Ramnath et al., "A Systematic Survey of Automatic Prompt Optimization Techniques," in Proc. EMNLP, 2025.

[2] L. Ngweta et al., "Towards LLMs Robustness to Changes in Prompt Format Styles," in Proc. NAACL Student Research Workshop, 2025.

[3] T. Zehle et al., "CAPO: Cost-Aware Prompt Optimization," in Proc. AutoML Conference, 2025.

[4] Y. Zhou et al., "Large Language Models Are Human-Level Prompt Engineers," in Proc. ICLR, 2023.

[5] C. Yang et al., "Large Language Models as Optimizers," in Proc. ICLR, 2024.

[6] H. Jiang et al., "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in Proc. EMNLP, 2023.

[7] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in Proc. ICLR, 2022.

[8] M. Conover et al., "Free Dolly: Introducing the World's First Truly Open Instruction-Tuned LLM," Databricks, 2023 (databricks-dolly-15k).

[9] S. Chaudhary, "Code Alpaca: An Instruction-following LLaMA Model for Code Generation," GitHub repository, 2023.

[10] Qwen Team, "Qwen2.5 Technical Report," arXiv:2412.15115, 2024.

[11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP-IJCNLP, 2019.

[12] K. L. Gwet, "Computing inter-rater reliability and its variance in the presence of high agreement," British Journal of Mathematical and Statistical Psychology, vol. 61, no. 1, pp. 29-48, 2008.

[13] A. R. Feinstein and D. V. Cicchetti, "High agreement but low kappa: I. The problems of two paradoxes," Journal of Clinical Epidemiology, vol. 43, no. 6, pp. 543-549, 1990.

[14] J. L. Fleiss, "Measuring nominal scale agreement among many raters," Psychological Bulletin, vol. 76, no. 5, pp. 378-382, 1971.

[15] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. ICML, 2021.

[16] R. Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in Proc. CVPR, 2022.

[17] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[18] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in Proc. ICLR, 2019.

[19] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. Chapman & Hall/CRC, 1993.

[20] F. Wilcoxon, "Individual Comparisons by Ranking Methods," Biometrics Bulletin, vol. 1, no. 6, pp. 80-83, 1945.

10. Scholarly contribution

Research paper

A research paper based on this project (the three-stage optimizer and its evaluation of input and output tokens on held-out data) is in preparation.